CallForge — User Manual

A targeted-capture germline analysis pipeline (FASTQ → annotated joint callset → burden)

Bright Adu

Genomics & Bioinformatics Core
Noguchi Memorial Institute for Medical Research
University of Ghana, Legon, Ghana

2026-06-14

Contents

1 1. Introduction **3**

2 2. Concepts **4**

3 3. Pipeline architecture **5**

4 4. Installation **7**
4.1 4.1 Local install (personal computer) . . . 7
4.2 4.2 Shared HPC install (SLURM + Apptainer/Singularity) . . . 7
4.3 4.3 The `callforge` command (unified CLI) . . . 8
4.4 4.4 How CallForge uses conda environments . . . 9

5 5. Inputs **10**
5.1 5.1 Sample sheet (CSV) — `--input` . . . 10
5.2 5.2 Target BED & gene metadata — CallForge runs without CaptureForge . . . 12
5.3 5.3 Reference — `--genome_fasta` . . . 14

6 6. Annotation databases **15**
6.1 6.1 The databases . . . 15
6.2 6.2 The genomic-scope gate (fail-loud) . . . 15
6.3 6.3 Contig-naming / chr-reconciliation . . . 16
6.4 6.4 Fetching databases — `bin/fetch_annotation_dbs.sh` . . . 16
6.5 6.5 Resource auto-discovery . . . 16
6.6 6.6 Bring-your-own / non-human databases . . . 16

7 7. Configuration **17**
7.1 7.1 Compute resources . . . 18
7.2 7.2 Parameter Reference (complete) . . . 18

8 8. Running — step-by-step scenarios **24**
8.1 8.0 How to invoke CallForge (command hierarchy) . . . 24
8.2 8.1 Quickstart (test profile) . . . 25
8.3 8.2 Full human cohort (primary scenario) . . . 25
8.4 8.3 Callset-only (no phenotype) . . . 26
8.5 8.4 Burden / association run (with phenotype) . . . 26
8.6 8.5 A different panel . . . 26
8.7 8.6 A different organism . . . 26
8.8 8.7 HPC run . . . 27
8.9 8.8 Choosing engine alternatives . . . 27
8.10 8.9 GIAB validation run . . . 27
8.11 8.10 Stage subcommands (run one stage standalone) . . . 27

9 9. Quality control **31**
9.1 9.1 Read & alignment QC (Stages 1–4) . . . 31
9.2 9.2 Per-sample QC gate & quarantine (Stage 5) . . . 31

9.3 9.3 Cohort QC dashboard & MultiQC (Stage 15) 36

10 10. Stage details & their plots **37**
10.1 10.1 SNV/indel calling & filtering (Stages 6–7) 37
10.2 10.2 CNV, STR, paralog (Stages 8–10) 37
10.3 10.3 Annotation (Stage 11) 37
10.4 10.4 Cohort QC (Stage 12) 37
10.5 10.5 GIAB validation (Stage 13) 37
10.6 10.6 Burden (Stage 14) 37

11 11. Outputs explained **49**

12 12. Interpreting results **50**

13 13. Real-run checklist **51**

14 14. Troubleshooting **52**

15 15. Extending CallForge **53**

16 16. Reproducibility & provenance **54**

17 17. Glossary & FAQ **55**

18 18. Citation & the CaptureForge ↔ CallForge pairing **56**

1. Introduction

CallForge is a disease- and organism-agnostic **Nextflow DSL2** pipeline that turns demultiplexed paired-end FASTQs from a hybridization-capture panel into a high-quality, joint-genotyped, **annotated** callset spanning **SNV/indel, CNV, STR and paralog-aware** variants, and carries through to a **rare-variant burden / association** layer.

CallForge is the **analysis counterpart to CaptureForge** (the probe-design pipeline) and **consumes CaptureForge's handoff**: the as-built target `final_covered_targets.bed` plus per-gene metadata (design class, CNV callability, paralog flags, STR loci, burden groups). That design-time knowledge drives and labels the matching analysis steps — the **CaptureForge → CallForge interlock**.

Its immediate use is a 59-gene human malaria-susceptibility panel on African-ancestry dried-blood-spot (DBS) samples, but it generalizes to **any panel, cohort, and (where feasible) organism** by changing inputs and configuration only — no code edits.

Who it is for. Bioinformaticians and genomics-core staff running targeted germline analysis who need a reproducible, QC-gated workflow with honest reporting. The pipeline emphasizes **QC at every stage with captioned plots**, **fail-loud invariants** (reference and annotation-database scope), and a **flag-and-quarantine** QC gate that never silently drops data.

> **Status note used throughout this manual.** CallForge is implemented and verified end-to-end on a small synthetic `test` profile. Where a number is a property of that synthetic data (not of a real cohort), the text and figure captions say so explicitly — *machinery-proven, not data quality.* See §11 (Interpreting results) and §12 (Real-run checklist).

2. Concepts

Targeted-capture germline analysis. A hybridization-capture panel enriches sequencing for a defined set of target regions (here, CaptureForge's `final_covered_targets.bed`). Reads are aligned to a reference, duplicates marked, and variants called *jointly* across the cohort so that genotypes — including homozygous-reference calls — are consistent and comparable across samples.

Joint genotyping. Per-sample gVCFs (GATK HaplotypeCaller) are combined (GenomicsDBImport or CombineGVCFs) and genotyped together (GenotypeGVCFs). Joint genotyping yields a square cohort matrix and well-calibrated genotypes, which matter for downstream cohort QC and burden testing.

The four variant classes (why each needs its own caller).

Class	Caller	Why a dedicated method
SNV / indel	GATK HaplotypeCaller (or DeepVariant)	local re-assembly of haplotypes
CNV	CNVkit (or GATK gCNV)	read-depth ratios vs a panel-of-normals, not per-base genotypes
STR	ExpansionHunter	repeat-aware genotyping from spanning/flanking reads
paralog-aware	`paralog_flag.py` confidence layer	multi-mapping ambiguity is a *confidence* problem, not a new caller

The CaptureForge → CallForge interlock. CaptureForge records, per gene, what it could and could not design well. CallForge consumes that:

- **CNV callability** — depth-callable genes (e.g. GYPC, HP) vs breakpoint-blind genes (e.g. CR1, CFH) label every CNV call's confidence.
- **STR catalog** — STR-class targets seed the ExpansionHunter variant catalog.
- **Paralog flags** — paralog-ambiguous genes (e.g. CD209, CASP1, CR1, HP, CFH) drive the paralog confidence layer.
- **Burden groups** — collapse units for gene-set burden testing.

The reference invariant. CallForge aligns to the **same no-alt GRCh38 primary assembly CaptureForge designed against**, and asserts it fail-loud: no alt/decoy contigs, and the target BED's build must match the reference (see §9, §13).

Rare-variant burden testing. Individually rare variants are aggregated per gene (and per gene-set) and tested for association with phenotype, adjusting for covariates and ancestry. CallForge filters to **rare + functional** variants, collapses by **gene** and **CaptureForge burden group**, and runs a gene-burden test (§7, §10).

3. Pipeline architecture

CallForge runs **16 stages (0–15)**. Each stage writes metrics (JSON/TSV), emits captioned plots (PNG+SVG), and feeds MultiQC plus a self-contained cohort QC dashboard. The diagram below is the authoritative map; external inputs (dashed = drives/labels downstream stages) feed in from the side.

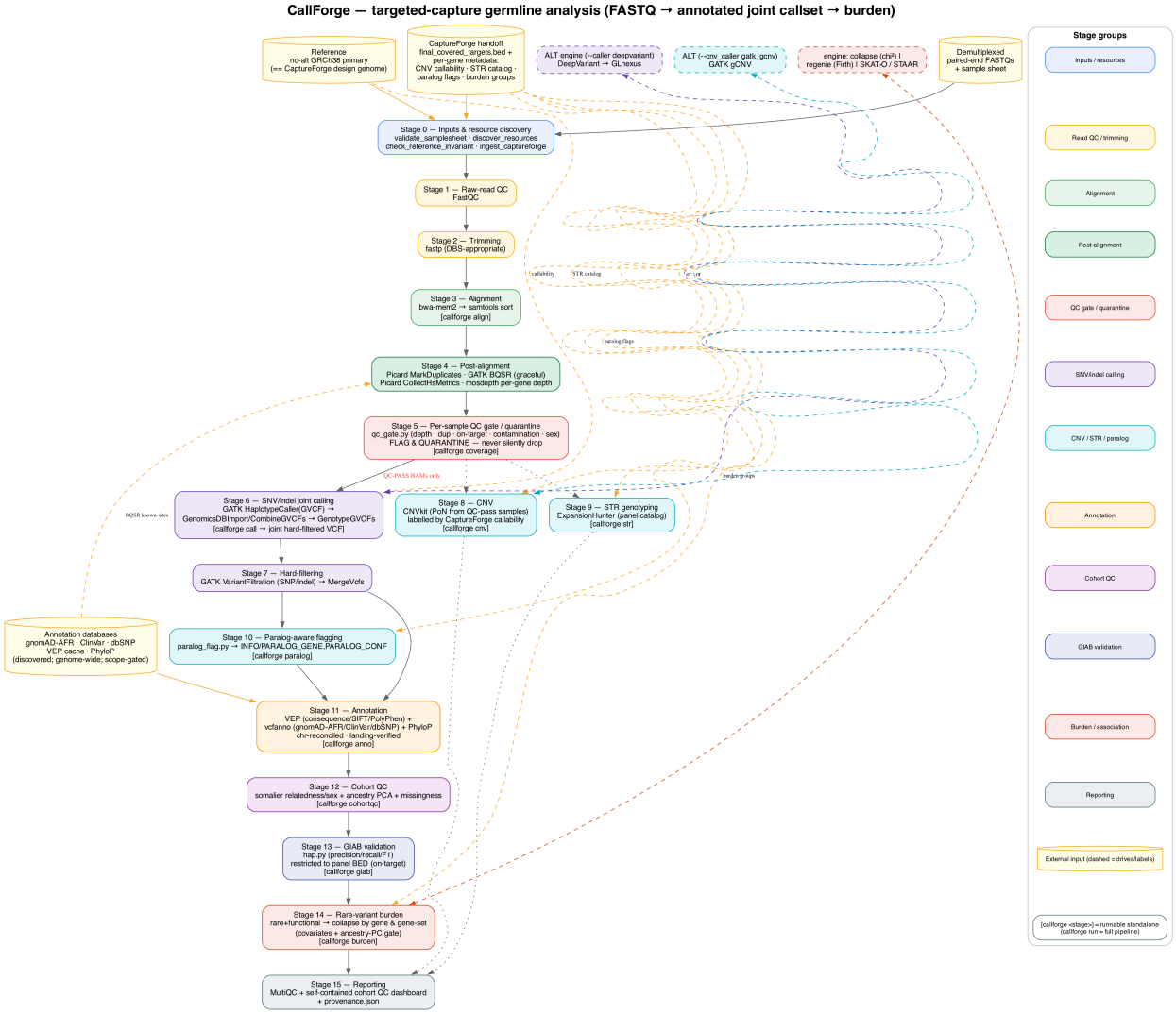


Figure 3.1: CallForge pipeline DAG — stages colour-coded by group; external inputs (reference, CaptureForge handoff, annotation databases, FASTQs) feed in; alternative engines (DeepVariant+GLnexus, gatk_gcnv, collapse/regenie/SKAT-O) shown as dashed branches.

Stage walkthrough (real inputs → outputs).

#	Stage	Key tool(s)	Output dir + key file(s)
0	Inputs & resource discovery	validate_samplesheet, discover_resources, ingest_captureforge	stage0_inputs/ — manifest, invariant, gene_metadata.tsv
1	Raw-read QC	FastQC	stage1_rawqc/
2	Trimming	fastp	stage2_trim/
3	Alignment	bwa-mem2, samtools sort	stage3_align/
4	Post-alignment	Picard MarkDuplicates, GATK BQSR, CollectHsMetrics, mosdepth	stage4_postalign/ — HsMetrics, mosdepth per-gene depth
5	Per-sample QC gate	qc_gate.py	stage5_qc_gate/ — qc_pass.txt, qc_quarantine.txt
6	SNV/indel joint calling	GATK HaplotypeCaller, GenomicsDBImport, GenotypeGVCFs	stage6_calling/ — joint.vcf.gz
7	Hard-filtering	GATK VariantFiltration, MergeVcfs	stage7_filter/ — joint.filtered.vcf.gz
8	CNV	CNVkit	stage8_cnv/ — cnv_calls.tsv (with callability)
9	STR	ExpansionHunter	stage9_str/ — str_calls.tsv
10	Paralog-aware	paralog_flag.py	stage10_paralog/ — paralog.annotated.vcf.gz
11	Annotation	VEP, vcfanno, PhyloP	stage11_annotation/ — annotated.vcf.gz, variants.flat.tsv
12	Cohort QC	somalier	stage12_cohortqc/
13	GIAB validation	hap.py	stage13_giab/
14	Burden / association	collapse / regenie / SKAT-O	stage14_burden/ — burden_results.tsv
15	Reporting	MultiQC, dashboard, provenance	stage15_report/

Nextflow / profile model. `main.nf` calls one subworkflow (`subworkflows/callforge.nf`) that wires modules in `modules/`. Behaviour is set by **profiles** composed on the command line: an execution profile (`test`, `local`, `hpc_slurm`) plus an environment engine (`conda`, `docker`, `apptainer`, `singularity`). Each process declares its own pinned conda env (`env/*.yml`) or a pinned container (VEP, hap.py, DeepVariant), so `-profile ...,conda`, `,apptainer`, or `,singularity` builds exactly what each stage needs.

Engine options (see §7 for full defaults): SNV/indel `--caller gatk|deepvariant`; joint method `--joint_method genomicsdb|combinegvcfs`; CNV `--cnv_caller cnvkit|gatk_gcnv`; CNVkit segmentation `--cnv_segment_method cbs|haar|hmm`; burden `--burden_engine regenie|skat|collapse`.

4. Installation

CallForge needs **Nextflow (>=23.10)** and either conda/mamba or a container engine. Each stage runs in its own pinned environment (`env/*.yml`) or pinned container, so you do not install tools by hand — `-profile conda` (or `,apptainer`) builds each environment on first use.

4.1 4.1 Local install (personal computer)

Common steps.

```
# 1. Nextflow (Java 17+ required)
curl -s https://get.nextflow.io | bash && sudo mv nextflow /usr/local/bin/
# or: mamba create -n nextflow -c bioconda 'nextflow>=23.10'

# 2. obtain CallForge and enter the repository
cd /path/to/callforge

# 3. core environment (others build on demand)
mamba env create -f env/callforge.yml
```

macOS (Apple Silicon). Some bioconda tools lack `osx-arm64` builds. Two options:

```
# (a) Rosetta-emulated x86 env (documented path; the report flags emulated
 ↪  stages)
CONDA_SUBDIR=osx-64 mamba env create -f env/callforge.yml

# (b) native arm64 - works for most tools, but pin mosdepth >= 0.3.11
#     (mosdepth 0.3.8 has no arm64 build; 0.3.8 + Rosetta is the alternative)
```

VEP and hap.py are run from their **official containers** (robust across platforms); ensure Docker is available and use `-profile <exec>,docker`.

Linux. Native; `mamba env create -f env/*.yml` resolves cleanly, or use the container route.

Container route. Build the core image once and run containerized:

```
docker build -t callforge:0.1.0 -f env/Dockerfile .
# HPC image (same .sif, either engine):
#   SingularityCE: sudo singularity build callforge.sif env/callforge.def
#                  (unprivileged: singularity build --fakeroot … needs
 ↪  /etc/subuid+subgid)
#   Apptainer:     apptainer build callforge.sif env/callforge.def
```

4.2 4.2 Shared HPC install (SLURM + Apptainer/Singularity)

The production cohort run is intended for the cluster (native linux-64; no emulation). The `hpc_slurm` profile uses the SLURM executor and runs tasks inside one container image — under **either Apptainer or SingularityCE** (compose `hpc_slurm,apptainer` or `hpc_slurm,singularity`; both run the same `.sif`), or shared conda (`hpc_slurm,conda`). This

is a brief orientation; **docs/hpc_deployment.md is the complete, authoritative procedure** (build → stage references → configure → run → troubleshoot). All paths here are site **examples**.

```
# 1. build the core image (Docker is usually forbidden on clusters)
apptainer build callforge.sif env/callforge.def          # or: singularity build
 ↪  …

# 2. run - wire the image to every task with --container_image, and bind the root
 ↪  that
#    holds your references (auto-mounts cover only $HOME/tmp/CWD).
 ↪  VEP/hap.py/DeepVariant
#    pull their own official images automatically.
export SINGULARITY_BIND=/hpc        # APPTAINER_BIND on an Apptainer site;
 ↪  survives env-scrub
callforge run -profile hpc_slurm,singularity -params-file site.params.yaml \
  --container_image /shared/apps/callforge/callforge.sif \
  --input samplesheet.csv --slurm_partition <queue> --scratch_dir /scratch/$USER
```

Two guards fail the run **loudly at launch** rather than mid-run: a container engine with no `--container_image` (which would silently fall back to host tools), and `hpc_slurm` with no `--slurm_partition` (no `normal` default — discover yours with `sinfo -s`).

The recommended front door is the **`callforge-run` wrapper** with a sourced site env file (no module system required) or an **Lmod** module — see `docs/hpc_deployment.md` §5. It puts `container_image`, `bind_paths`, `slurm_partition`, and the references in the params file / env once, so a run collapses to:

```
callforge-run -params-file site.params.yaml --input samplesheet.csv --outdir
 ↪  results
```

`conf/hpc_slurm.config` sets a shared container-image cache and conda-env cache so each is built/pulled once cluster-wide; `--slurm_partition` / `--slurm_account` / `--slurm_queue_opts` / `--scratch_dir` are overridable. Heavy steps (`index`, `align`, `large` labels) get bigger slots (16 cpu / 48–64 GB).

4.3 4.3 The `callforge` command (unified CLI)

CallForge ships a small console command, **`callforge`**, that is a friendly front door to the existing entry points. Install it from the repo with an editable pip install (it is stdlib-only — no heavy dependencies):

```
cd /path/to/callforge
pip install -e .
# registers `callforge` on PATH (in the active environment's bin/)
callforge --help        # list subcommands
callforge --version     # 0.1.0
```

Subcommands:

- `callforge run …` — run the full pipeline (thin wrapper over `nextflow run`; §8.0).
- `callforge samplesheet …` — scaffold/assemble a sample sheet (§5.1).
- `callforge resources …` — resource discovery / genomic-scope checks (`discover_resources.py`).

The CLI is **purely additive**: the raw forms keep working unchanged (`python3 bin/init_sample_sheet.py …`, `nextflow run main.nf …`). On the cluster, the shared conda env under `/hpc/opt/conda/envs/` plus the Lmod module/wrapper (§4.2) put `callforge` on `PATH` cluster-wide, so users run `callforge run …` without a manual install.

> Stage-level subcommands (e.g. `callforge cnv`, `callforge anno`) run a single stage standalone — each maps to `nextflow run main.nf --stage <name> …` (a `--stage` param, not `-entry`), reusing the same module as the full pipeline. See §8.10 for the full stage-subcommand reference.

4.4 4.4 How CallForge uses conda environments

CallForge deliberately uses **several small, isolated per-stage conda environments** rather than one big environment:

- `callforge` (core: alignment/QC/calling/filter), `callforge-cnv`, `callforge-str`, `callforge-annotate` (VEP/vcfanno), `callforge-cohortqc`, `callforge-burden`, `callforge-happy`, `callforge-deepvariant` — one per `env/*.yml`.

Different stages need **conflicting toolchains** (CNV vs STR vs VEP/Perl vs GATK/Java), and isolating them avoids dependency clashes — co-installing the CNV and STR tools into one shared environment corrupted it during development. This is the same pattern nf-core uses.

- **You never activate these by hand.** With `-profile conda` (or `,apptainer`), **Nextflow activates the correct environment/container for each process automatically**; you run one command (`callforge run …`).
- **Runtime vs dev-only.** The envs above are the *runtime contract* — they are declared by the pipeline stages and ship with it. A couple of environments are **dev-only byproducts** and are **not** part of that contract: the graphviz env used only to render this manual's DAG, and Nextflow's own runtime env.
- The unified `callforge` CLI is just a friendly front door — these isolated per-stage envs continue to work underneath it, invisibly.

5. Inputs

5.1 5.1 Sample sheet (CSV) — `--input`

Columns: `sample_id, fastq_1, fastq_2, sex, phenotype, covariate_*, batch`. Example:

```
sample_id,fastq_1,fastq_2,sex,phenotype,covariate_age,batch
S001,/data/fq/S001_R1.fastq.gz,/data/fq/S001_R2.fastq.gz,F,case,41,b1
S002,/data/fq/S002_R1.fastq.gz,/data/fq/S002_R2.fastq.gz,M,control,55,b1
HG002,/data/fq/HG002_R1.fastq.gz,/data/fq/HG002_R2.fastq.gz,M,control,30,b2
```

- `sample_id` unique, `[A-Za-z0-9._-]`; `fastq_1`/`fastq_2` are the paired-end reads (validated to exist and differ). `sex` is `M/F/U` (free tokens normalized).
- **`phenotype` is required only for the burden layer** (case/control or quantitative). Without it, the pipeline runs through the annotated callset and **skips burden with a message** (`burden_meta.json` records `status: skipped_no_phenotype`).
- `covariate_*` columns (any number) become burden covariates; `batch` is carried for QC.

Validation (`validate_samplesheet.py`) writes `samplesheet.valid.csv` and `samplesheet_summary.json` (with `burden_eligible`).

Where the sample sheet comes from. It is **not** a CaptureForge output — the design pipeline never saw your samples or their phenotypes. The two halves of the sheet have two different origins:

- `sample_id, fastq_1, fastq_2` come from your **sequencing core's demultiplexing** (the FASTQ files and their names).
- `sex, phenotype, covariate_*, batch` come from **your study / clinical metadata**.

The **phenotype is irreducibly yours**: CallForge never invents, guesses, or defaults it. The helper below only *pairs FASTQs* and *joins metadata you supply*; anything it cannot resolve it **reports** rather than fills.

Three ways to build one. The commands below use the unified CLI (`callforge samplesheet …`); the raw form `python3 bin/init_sample_sheet.py …` works identically.

1 — Hand-author the CSV. Write the schema above directly (absolute FASTQ paths). Fine for a handful of samples.

2 — Scaffold from a FASTQ directory (Mode A). Pairs R1/R2 by the usual Illumina conventions and infers `sample_id` from the filename, leaving the metadata columns blank for you to complete:

```
callforge samplesheet \
    --fastq-dir /data/fq \
    --out sample_sheet.csv
# then open sample_sheet.csv and fill sex / phenotype / covariate_* / batch
```

*3 — Scaffold FASTQs **and** join a metadata CSV* (Mode B) — the complete sheet in one step. Provide a clinical CSV keyed on `sample_id` (override with `--metadata-id-col`):

```
callforge samplesheet \
    --fastq-dir /data/fq \
    --metadata clinical.csv \
```

```
    --metadata-id-col sample_id \
    --keep-extra-cols \
    --out sample_sheet.csv
```

Mode C — map your core's manifest. If the core already gave you a `sample_id,fastq_1,fastq_2` export (a LIMS manifest), join metadata straight onto it without re-scanning a directory:

```
callforge samplesheet \
    --fastq-csv lims_manifest.csv \
    --metadata clinical.csv \
    --out sample_sheet.csv
```

Useful flags: `--recursive` (walk sub-directories), `--r1-pattern`/`--r2-pattern` (custom read markers), `--id-regex` (custom `sample_id` extraction), `--strip-suffixes` / `--lowercase-ids` (reconcile minor ID naming differences), and `--overwrite`. The helper **only reads** the FASTQ/metadata files — it never moves or modifies them.

Worked examples (input → produced sheet → checks). A FASTQ directory with four pairs (`CTRL`, `LOWQC`, `S1`, `S2`) is the input for Modes A and B below.

Mode A — only the sequencing columns are filled; `sex, phenotype, batch` are present but **blank** for you to complete (add `covariate_*` columns by hand or via a Mode-B join):

```
sample_id,fastq_1,fastq_2,sex,phenotype,batch
CTRL,/data/fq/CTRL_R1.fastq.gz,/data/fq/CTRL_R2.fastq.gz,,,
LOWQC,/data/fq/LOWQC_R1.fastq.gz,/data/fq/LOWQC_R2.fastq.gz,,,
S1,/data/fq/S1_R1.fastq.gz,/data/fq/S1_R2.fastq.gz,,,
S2,/data/fq/S2_R1.fastq.gz,/data/fq/S2_R2.fastq.gz,,,
```

Mode B — given this clinical CSV (note: no `LOWQC` row, and a stray `S07`):

```
sample_id,sex,phenotype,covariate_age,batch
CTRL,F,control,30,b1
S1,M,case,45,b1
S2,F,control,52,b2
S07,F,case,41,b2
```

the produced sheet joins the metadata in (the `covariate_age` column is carried through); `LOWQC` stays blank because it had no clinical row:

```
sample_id,fastq_1,fastq_2,sex,phenotype,batch,covariate_age
CTRL,/data/fq/CTRL_R1.fastq.gz,/data/fq/CTRL_R2.fastq.gz,F,control,b1,30
LOWQC,/data/fq/LOWQC_R1.fastq.gz,/data/fq/LOWQC_R2.fastq.gz,,,,
S1,/data/fq/S1_R1.fastq.gz,/data/fq/S1_R2.fastq.gz,M,case,b1,45
S2,/data/fq/S2_R1.fastq.gz,/data/fq/S2_R2.fastq.gz,F,control,b2,52
```

Mode C — given a LIMS manifest (`lims_manifest.csv`):

```
sample_id,fastq_1,fastq_2
S1,/data/fq/S1_R1.fastq.gz,/data/fq/S1_R2.fastq.gz
S2,/data/fq/S2_R1.fastq.gz,/data/fq/S2_R2.fastq.gz
```

and a clinical CSV whose `S02` is a typo for `S2`, the join produces:

```
sample_id,fastq_1,fastq_2,sex,phenotype,batch
```

```
S1,/data/fq/S1_R1.fastq.gz,/data/fq/S1_R2.fastq.gz,M,case,
S2,/data/fq/S2_R1.fastq.gz,/data/fq/S2_R2.fastq.gz,,,
```

— `S2` is left blank and the report flags `S02` as metadata-without-sequence with the near-miss hint `[closest FASTQ id: S2]` (re-run with `--strip-suffixes`/`--lowercase-ids`, or fix the typo, to make it join).

The reconciliation report. Alongside the CSV the helper writes `<out>_report.txt` and prints it. It explicitly lists every join discrepancy so a broken sheet cannot slip through silently:

- **orphan FASTQs** — an R1 or R2 with no mate (or no read marker at all);
- **sequence without metadata** — FASTQ paired but no clinical row;
- **metadata without sequence** — a clinical row with no FASTQ, *with the closest FASTQ `sample_id` as a near-miss hint* (e.g. `S02 [closest FASTQ id: S2]`);
- **duplicate `sample_id`** (in the FASTQs or the metadata);
- **rows missing a required field** after assembly;
- **blank phenotype** — warned (burden needs it) but **never auto-filled**;
- a one-line summary: *N written, N fully-joined, N sequence-only, N metadata-only, N orphan FASTQs.*

The helper **exits non-zero** on a *hard* problem — a duplicate `sample_id`, a row missing a required field, or `fastq_1 == fastq_2` — and in that case writes **no** sheet. Soft problems (orphans, unmatched IDs, blank phenotype) print as warnings but do not fail, so you still get a sheet to finish by hand. Example report:

```
CallForge sample-sheet reconciliation report (mode B)
============================================================
summary: 4 sample(s) written, 3 fully-joined, 1 sequence-only,
         1 metadata-only, 0 orphan FASTQ(s)

[SEQUENCE WITHOUT METADATA (FASTQ paired, no clinical row)] (1)
  - LOWQC
[METADATA WITHOUT SEQUENCE (clinical row, no FASTQ)] (1)
  - S07  [closest FASTQ id: S2]
[BLANK PHENOTYPE (burden needs it - fill manually, never auto-filled)] (1)
  - LOWQC
```

Feed the result back through validation (`validate_samplesheet.py`, run automatically by Stage 0) before a full run. Remember: **phenotype is required only for the burden stage** — without it the pipeline still produces the annotated joint callset and simply skips burden.

5.2 5.2 Target BED & gene metadata — CallForge runs without CaptureForge

The minimum to run CallForge is FASTQs + a reference + a target BED. The whole spine — align → coverage/QC → call → annotation → cohort QC → GIAB → per-gene burden — needs nothing from CaptureForge. The CaptureForge **per-gene metadata is optional enrichment** that the *interpretation layer* (CNV / STR / paralog / gene-set burden) consumes; absent it, those features degrade gracefully rather than failing.

What each metadata field unlocks, and the behavior when absent:

Metadata field	Unlocks	When absent
`is_str_target` + `str_loci`	STR genotyping (ExpansionHunter catalog)	STR stage produces nothing / skips
`is_paralog`	paralog-aware flagging (`PARALOG_GENE/CONF`)	no paralog flags written
`is_cnv_target`	CNV calls labelled as CNV targets	CNV calls still made, just unlabelled
`cnv_callable` (+`_reason`)	CNV calls labelled `callable` / `breakpoint_blind_low_confidence`	CNV calls labelled `unknown_confidence`
`burden_group`	gene-**set** burden (collapse by group)	burden collapses **per-gene only**
`off_target_frac`, `low_coverage`	specificity / coverage QC annotations	left blank (not assessed)

`--target_bed` is always required (it defines the regions). The BED's column-4 name uses `GENE|class` (e.g. `ATP2B4|coding`, `GYPC|cnv`, `HMOX1|str`); classes seed the per-gene table.

The `gene_metadata.tsv` schema (one row per gene; produced by either on-ramp below):

```
gene  classes  n_intervals n_coding n_promoter n_anchor n_str n_cnv
 ↪  is_cnv_target cnv_callable cnv_callable_reason  is_str_target str_loci
 ↪  is_paralog off_target_frac low_coverage burden_group
```

A hand-authored **minimal example** (two genes: a paralog flag on `CD209`, a burden group on both, everything design-specific left at its safe default):

```
gene    classes n_intervals n_coding    n_promoter  n_anchor    n_str   n_cnv
 ↪  is_cnv_target   cnv_callable    cnv_callable_reason is_str_target   str_loci
 ↪  is_paralog  off_target_frac low_coverage    burden_group
CD209   coding  1   1   0   0   0   0   no  n/a     no      yes
 ↪  COMPLEMENT
CFH cnv,coding  2   1   0   0   0   1   yes unknown
 ↪  user_declared_not_design_derived    no      no          COMPLEMENT
```

Two on-ramps — both emit this identical schema:

1. **Generate it (no CaptureForge):** `callforge metadata` builds `gene_metadata.tsv` from a gene list and/or the target BED, plus your own declarations:

   ```
   callforge metadata \
       --bed /panel/targets.bed \
       --cnv-genes CFH,CR1 \
       --paralog-genes CD209,CFH \
       --str-loci str_loci.tsv \
       --burden-groups burden_groups.tsv \
       --out-tsv gene_metadata.tsv
   ```

   It writes a reconciliation report (genes in the BED with no metadata, declared genes absent from the BED with near-miss hints, duplicates) and **exits non-zero on hard problems** (duplicate gene, malformed STR locus). It **never invents** the fields it cannot know: `cnv_callable` is set to `unknown` (`cnv_callable_reason=user_declared_not_design_derived`), `off_target_frac`/`low_coverage` are left blank, and **`burden_group` is never auto-assigned**

(blank → per-gene burden), exactly as `phenotype` is never auto-filled (§5.1). Then pass it with `--gene_metadata gene_metadata.tsv` (with `--paralog_genes` / `--str_catalog` as needed).

2. **From CaptureForge:** point CallForge at the as-built handoff and `ingest_captureforge.py` derives the same table (including design-derived callability):

```
target_bed:
 ↪  /path/captureforge/results/full_run/stage9_qc/final_covered_targets.bed
captureforge_dir: /path/captureforge/results/full_run    # auto-finds
 ↪  metrics.json + baits.csv
paralog_genes:    "CD209,CASP1,CR1,HP,CFH"
```

The one genuine complement. A generic generator can reproduce most of the enrichment, but **not** CaptureForge's `cnv_callable` / breakpoint-blindness labels: those encode **capture-design-specific** knowledge (which CNV breakpoints the bait layout can actually resolve) that no gene list or annotation can recreate. `callforge metadata` honestly marks CNV callability `unknown`; CaptureForge's design-derived callability is its value-add. *(Auto-flagging paralogs/STRs from public databases is a planned future enhancement; today those flags are user-declared.)*

5.3 5.3 Reference — `--genome_fasta`

The **no-alt GRCh38 primary assembly the panel was designed against** (Ensembl-named, e.g. `1,2,…,22,X,Y,MT`). `--genome_build GRCh38`. CallForge builds `.fai`/`.dict`/bwa-mem2 index if absent and asserts the reference invariant (§9).

6. Annotation databases

Annotation is the most resource-sensitive stage and is treated carefully: CallForge **discovers** local resources, **validates their genomic scope** (fail-loud on region-limited databases), and **reconciles contig naming** so values are never silently missed. Nothing is downloaded unless a resource is missing **and** required **and** you opt in (`--allow_download`).

6.1 6.1 The databases

Database	Provides	Stage(s) / param	Scope
gnomAD (AFR)	population allele frequencies, incl. African-ancestry AF_afr	11 (vcfanno), 14 (rare filter); –gnomad_vcf, –gnomad_af_field	genome-wide; scope-gated
ClinVar	clinical significance (CLNSIG, CLNDN, CLNREVSTAT, …)	11; –clinvar_vcf	genome-wide; scope-gated
dbSNP	rsIDs (ID column)	11 (rsID), 4 (BQSR); –dbsnp_vcf, –known_sites	genome-wide; scope-gated
VEP cache	consequence, transcript, SIFT/PolyPhen, canonical/MANE	11; –vep_cache, –vep_release	species + assembly match
PhyloP	conservation score (bigWig)	11; –phylop_bw	genome-wide

Namespacing. gnomAD INFO fields are written back as `gnomAD_*` (e.g. `gnomAD_AF_afr`) so the **cohort's own `AF/AC/AN` are never overwritten** — the classic annotation enrichment bug is designed out.

6.2 6.2 The genomic-scope gate (fail-loud)

A region-subset database (e.g. a single-locus gnomAD) silently returns "no AF / no ClinVar" outside its region — which a downstream filter would misread as *rare-novel*, corrupting burden testing. `discover_resources.py` therefore validates, per annotation VCF, that its data **spans the target panel's contigs** (tabix-probed, chr/no-chr reconciled), checks the **BGZF-EOF marker** (rejects truncated downloads), and **fails loud** on a found-but-unfit database:

```
[discover_resources] UNFIT (found but region-limited): gnomad_vcf: data spans
 ↪  only
1/19 target contigs (frac=0.0526 < 0.9); region-subset DB - replace with
 ↪  genome-wide.
[discover_resources] FAIL: ... Refusing to under-annotate.
```

Tune with `--scope_min` (default `0.9`); override only with `--ignore_unfit_resources` (not advised) or `--allow_download`.

6.3 6.3 Contig-naming / chr-reconciliation

The design reference / callset is **Ensembl-numeric** (`1`), gnomAD and PhyloP are usually **chr-prefixed** (`chr1`), and NCBI ClinVar is Ensembl-numeric. CallForge reconciles naming **per resource**: the query is renamed to a resource's style only where it differs, then renamed back — and the annotation step then **verifies the values actually landed** (`verify_annotation.py`, fail-loud), so a contig-name mismatch can never silently drop an annotation.

6.4 6.4 Fetching databases — `bin/fetch_annotation_dbs.sh`

A resumable downloader for genome-wide gnomAD v4.1 + ClinVar (GRCh38):

```
# panel-slice mode (recommended for a targeted panel; a few hundred MB)
MODE=panel \
PANEL_BED=/path/final_covered_targets.bed \
OUTDIR=/path/annotation_db \
  bash bin/fetch_annotation_dbs.sh

# full genome-wide mode (very large; use on HPC with a fast network)
MODE=full DATASETS="genomes" bash bin/fetch_annotation_dbs.sh
```

Panel mode merges nearby intervals to cut HTTP round-trips, slices each chromosome with per-chromosome retry, and is resumable (re-run to continue, not restart). **dbSNP is not fetched by this script** — supply a genome-wide dbSNP separately (it is also a BQSR known-site).

6.5 6.5 Resource auto-discovery

`discover_resources.py` scans `--resource_dirs` (default `$HOME,/usr/local/share,/opt`), validates each resource (build, index, scope, integrity), and writes `stage0_inputs/resource_manifest.json`. Missing human-only resources are reported and the dependent step skips gracefully. Explicit `--vep_cache/--gnomad_vcf/--clinvar_vcf/…` overrides take precedence and are still validated.

6.6 6.6 Bring-your-own / non-human databases

CallForge is organism-generic: VEP runs by **`--species`** (point at that species' VEP cache), and vcfanno reads a **bring-your-own-databases TOML** (`--vcfanno_toml`). Human-only resources (gnomAD/ClinVar/BQSR known-sites) **skip gracefully when absent**. To register a new organism: set `--species`, `--genome_fasta` (that build), `--vep_cache`, and a `--vcfanno_toml` listing whatever frequency/clinical databases exist; leave the human DB params unset.

7. Configuration

Set parameters via `-params-file params.yaml` (see `params.example.yaml`) or `--key value`. Profiles set executor + resource ceilings: **`test`** (3–4 synthetic samples, minutes), **`local`** (full, one machine; `mac_local` is a deprecated alias), **`hpc_slurm`** (SLURM + Apptainer/SingularityCE/shared-conda).

Key tunables (real defaults).

Param	Default	Meaning
`--caller`	`gatk`	SNV/indel engine: `gatk` or `deepvariant`
`--joint_method`	`genomicsdb`	`genomicsdb` or `combinegvcfs`
`--ploidy`	`2`	HaplotypeCaller ploidy
`--cnv_caller`	`cnvkit`	`cnvkit` or `gatk_gcnv`
`--cnv_method`	`hybrid`	CNVkit `hybrid` (capture) or `amplicon`
`--cnv_segment_method`	`cbs`	`cbs` (R/DNAcopy), `haar` (pure-python), `hmm`
`--min_mean_target_depth`	`30`	QC gate: min mean target depth (×)
`--max_dup_rate`	`0.40`	QC gate: max duplication fraction
`--min_on_target`	`0.40`	QC gate: min on-target fraction
`--max_contamination`	`0.03`	QC gate: max VerifyBamID2 FREEMIX
`--enforce_sex_check`	`true`	quarantine on sex mismatch (when assessable)
`--snp_filter_expr`	GATK WGS/WES default	SNP hard-filter JEXL
`--indel_filter_expr`	GATK WGS/WES default	indel hard-filter JEXL
`--scope_min`	`0.9`	min fraction of target contigs a DB must span
`--known_sites`	`null`	BQSR known-sites (dbSNP + Mills + 1000G)
`--somalier_sites`	`null`	somalier sites VCF (1000G for the build)
`--str_catalog`	`null`	ExpansionHunter catalog (else built from STR targets)
`--burden_engine`	`regenie`	`regenie`, `skat`, or `collapse`
`--burden_af_max`	`0.01`	rare threshold on gnomAD-AFR AF
`--burden_csq`	LoF + missense + splice + inframe set	qualifying consequences

Param	Default	Meaning
`--n_ancestry_pcs`	4	ancestry PCs offered as covariates (PC-stability gate applies)
`--vep_image`	ensembl-vep 110	VEP container (tag below)
`--happy_image`	hap.py 0.3.12	hap.py container (tag below)

The container defaults are `ensemblorg/ensembl-vep:release_112.0` (VEP) and `jmcdani20/hap.py:v0.3.12` (hap.py).

> **Hard-filter thresholds are the GATK WGS/WES defaults.** Revisit them on real targeted data (panel Ti/Tv, pass rates, and QUAL/DP distributions guide tuning).

7.1 7.1 Compute resources

CallForge uses Nextflow's **per-process** resource model, not one global `--threads`: each stage declares the CPUs/memory it needs via a resource **label** (`small`/`medium`/`large`, plus `index`/`align`), and the tools receive those cores — e.g. `bwa-mem2 mem -t`, `samtools sort -@`, `fastp --thread`, `mosdepth -t`, `CNVkit -p`, `GenomicsDBImport --reader-threads`, `HaplotypeCaller --native-pair-hmm-threads`, `DeepVariant --num_shards`, `GLnexus --threads`, `VEP --fork`, `ExpansionHunter --threads` (all wired to the process `cpus`). A few tools are single-threaded by design (Picard MarkDuplicates/CollectHsMetrics, GATK GenotypeGVCFs).

Single-machine ceilings (the laptop knob). On the `local` profile, total concurrent use is capped by two params — set "use at most N cores / M GB", don't micromanage each tool:

Param	Default (`local`)	Meaning
`--max_cpus`	8	max total cores CallForge uses at once on the local machine
`--max_memory`	24.GB	max total memory used at once

```
# a 4-core / 8 GB laptop:
callforge run -profile local,conda -params-file params.full.yaml \
    --max_cpus 4 --max_memory 8.GB
```

These feed the `local` executor's total `cpus`/`memory`, and every per-stage request is **clamped** to them (`[request, ceiling].min()`), so no single task ever asks for more than the machine is allowed to use. The `hpc_slurm` profile instead requests resources **per SLURM job** (e.g. `index`/`align` get 16 cores) and `--max_cpus`/`--max_memory` bound per-job sizes.

Power-user escape hatches (Nextflow-native): override one process with `-process.cpus=N`, or supply a custom config with `-c my.config` (e.g. a `withName:` block to retune a specific process). The defaults live in `conf/base.config` (per-label) and each profile's `conf/*.config`.

7.2 7.2 Parameter Reference (complete)

Every CallForge parameter, its real default, whether it is required, and its **provenance** — where the value comes from in a deployed setup:

- **[install]** — set once per cluster in the site env file (`callforge-env.sh`: the `CALLFORGE_*` vars the wrapper turns into flags).
- **[site]** — set once per cluster in the params file (`-params-file`): references, databases, container image, queue.
- **[per-run]** — you provide every run (the minimal set).
- **[optional]** — has a sane default; set only to override.
- **[stage-dependent]** — only needed when that stage/engine/branch runs.

Parameter	What it is	Required?	Default	Provenance
`--input`	sample sheet CSV (`sample_id,fastq_1,fastq_2[,sex,phenotype,covariate_*,batch]`)	**yes** (full pipeline)	`null`	[per-run]
`--genome_fasta`	no-alt primary-assembly FASTA (same build/contig naming as the BED)	**yes**	`null`	[site]
`--target_bed`	panel BED (CaptureForge `final_covered_targets.bed` or any panel BED)	**yes**	`null`	[site]
`--panel_name`	label stamped on outputs	no	`callforge_panel`	[per-run]/[optional]
`--captureforge_dir`	CaptureForge `results/<run>` dir (auto-finds metrics.json + baits.csv)	no	`null`	[optional]
`--cf_metrics_json` / `--cf_baits_csv`	explicit CaptureForge artifacts	no	`null`	[optional]
`--gene_metadata`	pre-built per-gene metadata TSV (use **without** CaptureForge)	no	`null`	[optional]
`--paralog_genes`	comma list of paralog genes	no	`null`	[optional]
`--str_catalog`	ExpansionHunter catalog JSON (else built from STR targets)	no	`null`	[optional]
`--genome_build`	reference build label	no	`GRCh38`	[optional]
`--species`	VEP species	no	`homo_sapiens`	[optional]

Parameter	What it is	Required?	Default	Provenance
`--resource_dirs`	comma list scanned by resource discovery	no	`$HOME,/usr/local/share,/opt`	[site]
`--resource_manifest`	pre-built manifest JSON (skip discovery)	no	`null`	[site]
`--allow_download`	fetch a missing **required** DB	no	`false`	[optional]
`--vep_cache` / `--vep_release`	VEP cache dir / release	no (else discovered)	`null` / `110`	[site]
`--gnomad_vcf` / `--gnomad_af_field`	gnomAD VCF (prefer AFR) / source AF field	no	`null` / `AF_afr`	[site]
`--dbsnp_vcf` / `--clinvar_vcf` / `--phylop_bw`	annotation databases	no	`null`	[site]
`--known_sites`	BQSR known-sites (dbSNP + Mills + 1000G), comma list	no (else BQSR skipped)	`null`	[site]
`--vcfanno_toml`	bring-your-own vcfanno databases (organism-generic)	no	`null`	[site]
`--scope_min`	min fraction of target contigs a DB must span (else UNFIT, fail loud)	no	`0.9`	[optional]
`--ignore_unfit_resources`	proceed despite a region-limited DB (not advised)	no	`false`	[optional]
`--adapter_r1` / `--adapter_r2`	fastp adapters (null = auto-detect)	no	`null`	[optional]
`--fastp_qual` / `--fastp_min_len`	trim quality / min length	no	`20` / `30`	[optional]
`--min_mean_target_depth`	QC gate: min mean target depth (×)	no	`30`	[optional]

Parameter	What it is	Required?	Default	Provenance
`--max_dup_rate` / `--min_on_target` / `--max_contamination`	QC gate thresholds	no	`0.40` / `0.40` / `0.03`	[optional]
`--enforce_sex_check`	quarantine on sex mismatch	no	`true`	[optional]
`--caller`	SNV/indel engine: `gatk` | `deepvariant`	no	`gatk`	[optional]
`--ploidy` / `--joint_method`	HaplotypeCaller ploidy / `genomicsdb`|`combinegvcfs`	no	`2` / `genomicsdb`	[optional]
`--snp_filter_expr` / `--indel_filter_expr`	GATK hard-filter JEXL (WGS/WES defaults)	no	see config	[optional]
`--cnv_caller` / `--cnv_method` / `--cnv_segment_method`	CNVkit knobs	no	`cnvkit` / `hybrid` / `cbs`	[optional]
`--cnv_min_samples_pon`	min QC-pass samples to build a panel-of-normals	no	`5`	[optional]
`--str_motif`	fallback motif when building an STR catalog	no	`GT`	[optional]
`--paralog_min_mq`	MQ below which a paralog-region call is flagged	no	`50`	[optional]
`--vep_mode` / `--gtf` / `--vep_extra`	`cache`|`gtf`; GTF for gtf mode; extra VEP flags	no	`cache` / `null` / `''`	[optional]/[stage-dependent]
`--vep_image`	VEP container	no	`ensemblorg/ensembl-vep:release_112.0`	[optional]
`--somalier_sites`	somalier sites VCF (cohort QC)	no	`null`	[stage-dependent]
`--giab_control_id`	control `sample_id` in the callset to benchmark	no	`null`	[per-run]
`--giab_truth_vcf` / `--giab_truth_bed`	GIAB truth VCF / high-confidence BED	no	`null`	[site]
`--happy_image`	hap.py container	no	`jmcdani20/hap.py:v0.3.12`	[optional]

Parameter	What it is	Required?	Default	Provenance
`--run_burden`	run the burden stage (auto-skipped without a phenotype column)	no	`true`	[optional]
`--burden_engine`	`regenie` | `skat` | `collapse`	no	`regenie`	[optional]
`--burden_af_max` / `--burden_csq`	rare-AF threshold / qualifying consequences	no	`0.01` / LoF+missense set	[optional]
`--n_ancestry_pcs` / `--covariates`	ancestry PCs / covariate_* columns (''=all)	no	`4` / `''`	[optional]
`--container_image`	image every task runs in (.sif path | docker tag)	**yes if an engine is enabled**	`''`	[site]/[install]
`--bind_paths`	host path(s) to bind into containers for in-place refs	no	`''`	[site]/[install]
`--slurm_partition`	SLURM queue	**yes under `hpc_slurm`**	`null`	[site]
`--slurm_account` / `--slurm_queue_opts` / `--scratch_dir`	SLURM account / extra opts / scratch	no	`''` / `''` / `/tmp`	[optional]
`--max_cpus` / `--max_memory` / `--max_time`	per-profile compute ceilings	no	per profile	[optional]
`--outdir` / `--publish_mode`	results directory / publish mode	no	`results` / `copy`	[per-run]/[optional]

Stage-subcommand-only inputs (read only under `--stage`/`callforge <stage>`; ignored by the full `callforge run`): `--stage`, `--input_vcf`, `--input_bams`, `--fastq_1`/`--fastq_2`/ `--sample_id`, `--cohort_qc_json`, `--in_place`. See §8.10 and `callforge <stage> --help`.

> **Annotated, ready-to-edit templates.** `params.example.yaml` (local) and `conf/cluster.params.example.yaml` (cluster) carry these with site-example values; the `callforge run --help` epilog lists them inline with the same provenance tags.

7.2.1 Expanded vs. short run (the same run, two ways)

```
# expanded - every value on the command line (illustrative; you rarely type this)
```

```
callforge run -profile hpc_slurm,singularity \
    --container_image /shared/apps/callforge/callforge.sif \
    --slurm_partition compute --scratch_dir /scratch/$USER --bind_paths /hpc \
    --input samplesheet.csv \
    --genome_fasta /hpc/refs/callforge/GRCh38/GRCh38_noalt_primary.fa \
    --target_bed /hpc/refs/callforge/GRCh38/final_covered_targets.bed \
    --resource_dirs /hpc/refs/callforge/annotation_db \
    --caller gatk --burden_engine regenie --outdir results

# short - site values live in the env file + params file; the wrapper supplies
 ↪  image+bind
callforge-run -params-file site.params.yaml --input samplesheet.csv --outdir
 ↪  results
```

8. Running — step-by-step scenarios

All multi-flag commands use backslash continuation (one flag per line). On a local machine use `,conda` (and `,docker` for the VEP/hap.py containers); on the cluster use `,apptainer`.

8.1 8.0 How to invoke CallForge (command hierarchy)

There are three equivalent ways to launch CallForge — all run the *same* pipeline. Lead with the named command; drop to the raw forms only when you have not installed the CLI.

Tier 1 — the named command (recommended, after `pip install -e .`). `callforge run` is a thin wrapper over `nextflow run`; every flag after it is passed straight to Nextflow.

```
# runs the full CallForge pipeline (FASTQ -> annotated joint callset -> burden)
callforge run \
    -profile local,conda \
    -params-file params.full.yaml

# the same on an HPC cluster (SLURM + SingularityCE; or ,apptainer)
callforge run \
    -profile hpc_slurm,singularity \
    -params-file params.full.yaml
```

Tier 2 — from GitHub without cloning (Nextflow pulls the repo for you):

```
# runs CallForge straight from GitHub; version-pin with -r <tag/branch>
nextflow run aduton1000/callforge -r v0.1.0 \
    -profile hpc_slurm,singularity \
    -params-file params.full.yaml
```

The GitHub account is **aduton1000**. The repo is **currently private**, so this form works only for users with repo access until it is made public; always pin a version with `-r <tag/branch>` for reproducibility.

Tier 3 — from a cloned repo (developers):

```
# runs CallForge from inside the clone
nextflow run main.nf \
    -profile local,conda \
    -params-file params.full.yaml
```

`main.nf` **is** CallForge's entry script — it imports the subworkflows and wires the DAG, so `nextflow run main.nf` runs the whole pipeline, but only from inside the repo directory. `callforge run` (Tier 1) simply locates this `main.nf` for you and runs it from anywhere.

In every tier, `params.full.yaml` supplies the **reference** (`genome_fasta`), the **target BED + CaptureForge handoff** (`target_bed` / `captureforge_dir`), the **annotation resources** (resource directories), and the **sample sheet** (`input`).

8.1.1 Understanding profiles

A CallForge run composes **two** profiles with a comma — an **execution** profile and a **packaging** profile:

- **Execution** (where it runs): `local` = this single machine, `hpc_slurm` = a SLURM cluster. `local` works on **any single Linux or macOS machine (Windows via WSL2)** — it is *not* Mac-specific and *not* developer-only (the name `mac_local` is a deprecated alias kept for backward compatibility). `local` caps total resource use at `--max_cpus`/`--max_memory` (§Compute resources); `hpc_slurm` requests resources per SLURM job.
- **Packaging** (how dependencies are provided): `conda`, `docker`, `singularity` (SingularityCE), or `apptainer` — Nextflow activates the right per-stage env/container automatically (§4.4).

```
callforge run -profile local,conda          -params-file params.full.yaml # local
↪  machine, conda envs
callforge run -profile hpc_slurm,singularity -params-file params.full.yaml #
↪  SLURM cluster, container images (or ,apptainer)
```

`local,conda` = "run here, build each stage's conda env"; `hpc_slurm,singularity` (or `,apptainer`) = "submit SLURM jobs, run each stage in its container image". Mix to taste (e.g. `local,docker` for the VEP/hap.py containers on a workstation).

8.2 8.1 Quickstart (test profile)

```
# build a tiny fixture, then run all 16 stages of CallForge in minutes
bash test/make_test_data.sh
callforge run -profile test,docker
```

This builds a tiny self-contained fixture (mini-genome + synthetic reads) and runs all 16 stages in minutes. Use it to confirm your install.

8.3 8.2 Full human cohort (primary scenario)

```
# 1. write the sample sheet (§5.1) with phenotype if you want burden

# 2. confirm resources discover + pass the scope gate; fetch genome-wide
#    gnomAD/ClinVar if missing (§6.4); supply a genome-wide dbSNP

# 3. run CallForge (full pipeline)
callforge run \
  -profile local,conda \
  -params-file params.full.yaml

# 3'. or the cluster (SingularityCE; or ,apptainer)
callforge run \
  -profile hpc_slurm,singularity \
  -params-file params.full.yaml
```

`params.full.yaml` wires the reference, the CaptureForge handoff, resource directories, and engine choices; set `input:` to your cohort sample sheet. Inspect `results/stage5_qc_gate/qc_scorecard.tsv`,

then the annotated callset and the dashboard.

8.4 8.3 Callset-only (no phenotype)

Omit `phenotype` (or set `--run_burden false`). The pipeline runs through Stage 11 (annotated callset) + cohort QC + GIAB and **skips burden**; `burden_meta.json` records the skip. Everything else (annotated VCF, per-variant TSV, CNV/STR/paralog) is produced.

8.5 8.4 Burden / association run (with phenotype)

```
callforge run \
  -profile hpc_slurm,singularity \
  -params-file params.full.yaml \
  --run_burden true \
  --burden_engine regenie \
  --burden_af_max 0.01 \
  --n_ancestry_pcs 4 \
  --covariates covariate_age,covariate_sex
```

Burden filters to rare+functional variants, collapses by gene and CaptureForge burden group, and tests with covariates + ancestry PCs. The **ancestry-PC stability gate** drops the PCs and runs unadjusted (recorded in provenance) if there are too few PCA sites for the cohort — read `burden_results.tsv` (its `engine` column states the engine) with the QQ / Manhattan plots and the validity caveats (§11).

8.6 8.5 A different panel

Swap the CaptureForge handoff — no code change:

```
callforge run -profile local,conda \
  --input samplesheet.csv \
  --genome_fasta /ref/GRCh38_noalt.fa \
  --target_bed   /panelB/final_covered_targets.bed \
  --captureforge_dir /panelB/results/full_run
```

8.7 8.6 A different organism

```
callforge run -profile local,conda \
  --input samplesheet.csv \
  --genome_build <BUILD> \
  --genome_fasta /ref/<species>_noalt.fa \
  --target_bed /panel/final_covered_targets.bed \
  --species <vep_species> \
  --vep_cache /vep/<species> \
  --vcfanno_toml /db/<species>.toml \
  --run_burden false
```

Human-only resources (gnomAD/ClinVar/BQSR) skip gracefully. Minimum inputs: sample sheet, no-alt reference for that build, target BED, a VEP cache (or `--vep_mode gtf` with a `--gtf`), and a vcfanno TOML for any frequency/clinical databases you have.

8.8 8.7 HPC run

See §4.2 and the full procedure in `docs/hpc_deployment.md`. Compose `-profile hpc_slurm,singularity` (SingularityCE) or `hpc_slurm,apptainer` (both run the same `.sif`), wire the image with `--container_image`, set `--slurm_partition <queue>` (no default), and bind the references' root (`export SINGULARITY_BIND=/path` or `--bind_paths`). The `callforge-run` wrapper + a sourced site env file reduce a run to `callforge-run -params-file site.params.yaml --input samplesheet.csv --outdir results`.

8.9 8.8 Choosing engine alternatives

```
--caller deepvariant            # DeepVariant + GLnexus (container) instead of
↪ GATK
--cnv_caller gatk_gcnv          # GATK gCNV instead of CNVkit
--burden_engine skat            # SKAT-O / STAAR (R) instead of regenie
--burden_engine collapse        # chi-square collapsing (small-N /
↪ dependency-free)
```

> **Validate before production.** `DeepVariant+GLnexus`, `regenie`, and `SKAT-O/STAAR` are wired but have **not yet been executed end-to-end** (the synthetic test cohort is too small). Exercise them on a real/realistic cohort before relying on them. The tested engines are GATK/GenomicsDB (calling) and `collapse` (burden).

8.10 8.9 GIAB validation run

Add a GIAB control to the sample sheet and supply truth:

```
callforge run -profile local,conda -params-file params.full.yaml \
  --giab_control_id HG002 \
  --giab_truth_vcf /giab/HG002_v4.2.1_benchmark.vcf.gz \
  --giab_truth_bed /giab/HG002_v4.2.1_benchmark.bed
```

hap.py scores precision/recall/F1 **restricted to the panel BED** (on-target only), so off-target truth cannot distort the panel's measured sensitivity.

8.11 8.10 Stage subcommands (run one stage standalone)

Sometimes you only want to re-run **one** stage on its own inputs — re-annotate after an annotation-database update, re-run burden with new thresholds, re-call CNV — without repeating the whole pipeline. Each analytical stage is exposed as `callforge <stage>`, which runs `nextflow run main.nf --stage <stage> …` (a `--stage` param, not `-entry`: portable across Nextflow versions). **Every stage subcommand invokes the exact same module the full pipeline uses** — standalone and in-pipeline execution are the same code path, so results never depend on how the stage was launched.

Three things hold for every stage subcommand:

- **Non-destructive by default.** Output goes to a fresh `results/standalone/<stage>_<timestamp>/` so a re-run never overwrites the pipeline's `results/stageXX_*`. Override with `--outdir <dir>`, or opt into updating `results/` in place with `--in-place`.
- **Each stage emits its own report + provenance** in its stage dir: `<stage>_report.md` (key metrics + that stage's QC plots with captions) and `<stage>_provenance.json` (inputs + checksums, parameters, tool versions, git commit).
- **Inputs are validated fail-loud** before the stage runs (index/contig agreement, the reference invariant and DB scope gate where relevant, phenotype for burden), exiting non-zero on a mismatch.

`callforge <stage> --help` prints that stage's required + optional input files.

Key-output filenames below are relative to the stage's own output directory (`stageXX_…/`), e.g. `align` writes `stage4_postalign/`, `anno` writes `stage11_annotation/`. Each stage also writes `<stage>_report.md` + `_provenance.json`.

`callforge`	Required input files	Key output(s)	QC plots
`align` *(per-sample)*	`--input` or `--fastq_1/2`; `--genome_fasta`; `--target_bed`	`<sample>.analysis.bam`	mapping rate, insert size, dup rate
`coverage` *(per-sample → gate)*	`--input_bams`; `--genome_fasta`; `--target_bed`	`qc_scorecard.tsv`, `qc_pass.txt`, `qc_quarantine.txt`	on-target enrichment, coverage uniformity, per-gene depth, coverage cumulative, QC scorecard
`call` *(joint)*	`--input_bams` (QC-pass); `--genome_fasta`; `--target_bed`	`joint.filtered.vcf.gz`	variants per sample, Ti/Tv, het:hom, QUAL/DP, filter counts
`cnv` *(per-sample)*	`--input_bams`; `--genome_fasta`; `--target_bed`	`cnv_calls.tsv` (callability-labelled)	copy ratio, calls per sample, callability
`str` *(per-sample)*	`--input_bams`; `--genome_fasta`; `--target_bed`	`str_calls.tsv`	allele sizes, call rate, read support
`paralog` *(joint VCF)*	`--input_vcf`; `--target_bed`	`paralog.annotated.vcf.gz` (`PARALOG_GENE`/`CONF`)	paralog confidence
`anno` *(VCF)*	`--input_vcf`; `--genome_fasta`; `--target_bed`	`annotated.vcf.gz`, `variants.flat.tsv`	annotation landing, consequence dist., novel vs known, gnomAD-AFR AF, PhyloP
`cohortqc` *(cohort)*	`--input_bams`; `--input_vcf`; `--genome_fasta`; `--somalier_sites`; `--input`	`cohort_qc.json`	relatedness heatmap, ancestry PCA, sex check, missingness

callforge	Required input files	Key output(s)	QC plots
giab *(control)*	--input_vcf; --giab_control_id; --giab_truth_vcf/_bed; --target_bed; --genome_fasta	happy.summary.csv, happy.runinfo.json	precision/recall, F1
burden *(cohort)*	--input_vcf; --input (phenotype); --target_bed	burden_results.tsv	Q–Q, Manhattan

Notable **optional** inputs: `align --known_sites`; `call --caller deepvariant` / `--joint_method`; `cnv --cnv_method`/`--cnv_segment_method` (a different CNV *caller*, `gatk_gcnv`, is documented but not yet wired — same as the pipeline); `str --str_catalog`; `anno --vcfanno_toml`/`--species`; `burden --burden_engine`/`--burden_af_max`/`--burden_csq` and `--cohort_qc_json` (ancestry PCs from a prior `cohortqc` run). The CaptureForge metadata (callability, STR catalog, paralog regions, burden groups) is rebuilt from `--target_bed` + `--cf_metrics_json`/`--captureforge_dir` exactly as in-pipeline, so those labels appear on standalone outputs too. `paralog` uses the VCF's own `MQ` (it does **not** take BAMs); `cohortqc` needs **both** BAMs (somalier/sex) and the joint VCF (missingness/PCA).

8.11.1 Worked examples

1 — Re-annotate after an annotation-database update (e.g. a new ClinVar release). Point discovery at the updated database directory and re-run only Stage 11 on the existing paralog-flagged callset:

```
callforge anno \
    --input_vcf results/stage10_paralog/paralog.annotated.vcf.gz \
    --genome_fasta /refs/GRCh38_noalt.fa \
    --target_bed /captureforge/final_covered_targets.bed \
    --resource_dirs /refs/annotation_db_2025_06 \
    -profile local,conda
# -> results/standalone/anno_<timestamp>/stage11_annotation/annotated.vcf.gz
#    + variants.flat.tsv, anno_report.md, anno_provenance.json
```

2 — Re-run burden with a new AF cutoff, consequence set, and engine. Reuse the annotated callset; change only the thresholds/engine. Optionally reuse ancestry PCs from a prior `cohortqc` run:

```
callforge burden \
    --input_vcf results/stage11_annotation/annotated.vcf.gz \
    --input sample_sheet.csv \
    --target_bed /captureforge/final_covered_targets.bed \
    --burden_af_max 0.005 \
    --burden_csq
     ↪ 'frameshift_variant,stop_gained,splice_acceptor_variant,splice_donor_variant'
     ↪ \
```

```
    --burden_engine regenie \
    --cohort_qc_json results/stage12_cohortqc/cohort_qc.json \
    -profile local,conda
# -> results/standalone/burden_<timestamp>/stage14_burden/burden_results.tsv
#    (engine column stamps the method) + burden_report.md, burden_provenance.json
```

3 — Re-call CNV with different segmentation. Re-run Stage 8 on the same BAMs with a different CNVkit method/segmenter (the calls stay CaptureForge-callability-labelled):

```
callforge cnv \
    --input_bams 'results/standalone/align_*/stage4_postalign/*.analysis.bam' \
    --genome_fasta /refs/GRCh38_noalt.fa \
    --target_bed /captureforge/final_covered_targets.bed \
    --cf_metrics_json /captureforge/results/full_run/stage9_qc/metrics.json \
    --cnv_method hybrid --cnv_segment_method cbs \
    -profile local,conda
# -> results/standalone/cnv_<timestamp>/stage8_cnv/cnv_calls.tsv
#    (each call labelled callable | breakpoint_blind_low_confidence) +
 ↪  cnv_report.md
```

(The alternative SNV caller is available the same way: `callforge call --caller deepvariant …` runs DeepVariant + GLnexus instead of GATK; see §8.8.)

9. Quality control

CallForge's QC philosophy: **a plot at every applicable stage** (each with a one-line caption), gates that **flag and quarantine** rather than silently drop, and a single cohort QC dashboard plus MultiQC. The figures below are from the synthetic `test` run — they prove the machinery; real numbers come from a real cohort.

9.1 9.1 Read & alignment QC (Stages 1–4)

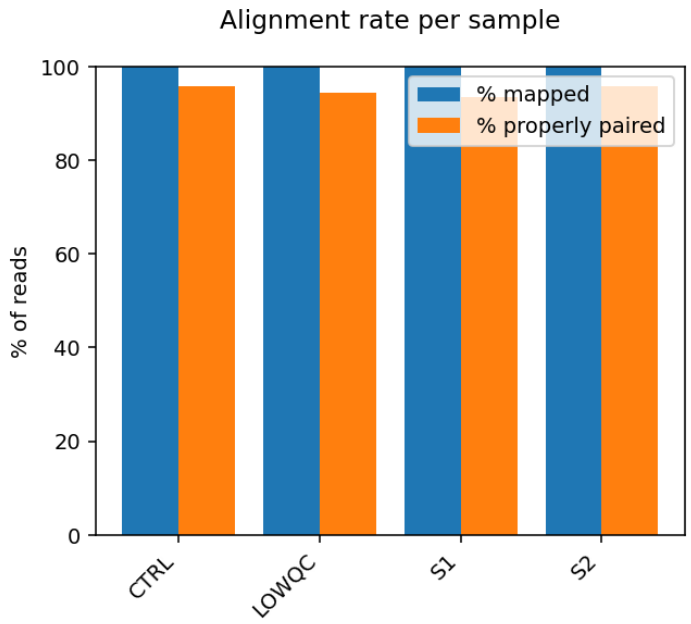


Figure 9.1: Percent of reads mapped and properly paired per sample (alignment success).

`CollectHsMetrics` (vs the target BED) gives on-target %, fold-enrichment, % target ≥10/20/30×, and uniformity (FOLD_80); mosdepth gives per-target and **per-gene depth** in the CaptureForge track style. Duplicates are marked by coordinate (correct for capture). BQSR runs when known-sites are available and **skips gracefully** otherwise.

9.2 9.2 Per-sample QC gate & quarantine (Stage 5)

`qc_gate.py` aggregates the per-sample metrics and applies thresholds (depth, dup, on-target, contamination, sex). Failing samples are **flagged and QUARANTINED** — written to `qc_quarantine.txt` with reasons, **excluded from joint calling and from burden counts**, but never silently dropped. Metrics that cannot be assessed (contamination without a VerifyBamID2 SVD panel; sex on an autosomal panel with no X/Y targets) are recorded **NA**, not failed.

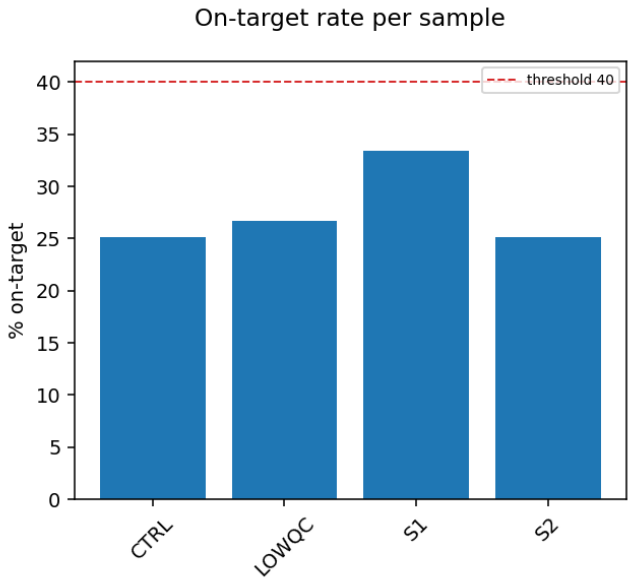


Figure 9.2: Percent of bases on the capture target per sample (capture specificity).

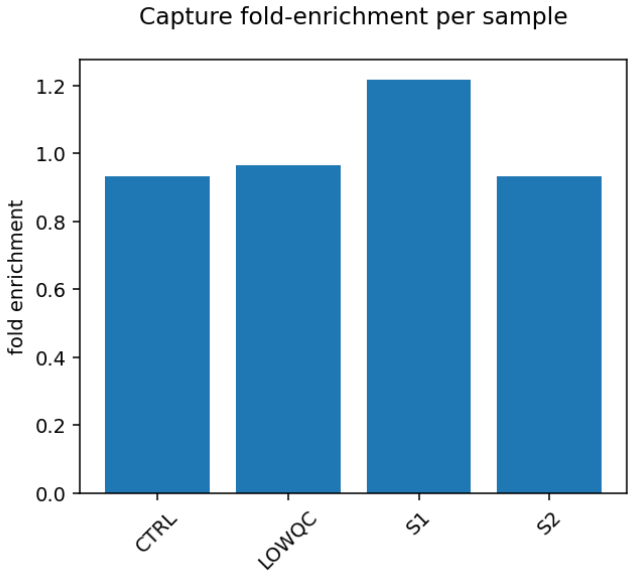


Figure 9.3: Fold-enrichment of on-target vs genome-average coverage (capture performance).

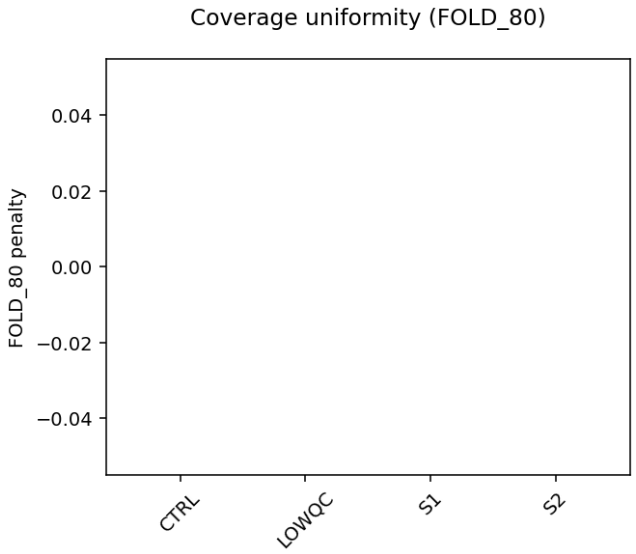


Figure 9.4: FOLD_80_BASE_PENALTY: fold extra sequencing for 80% of targets to reach the mean (1.0 = perfectly uniform).

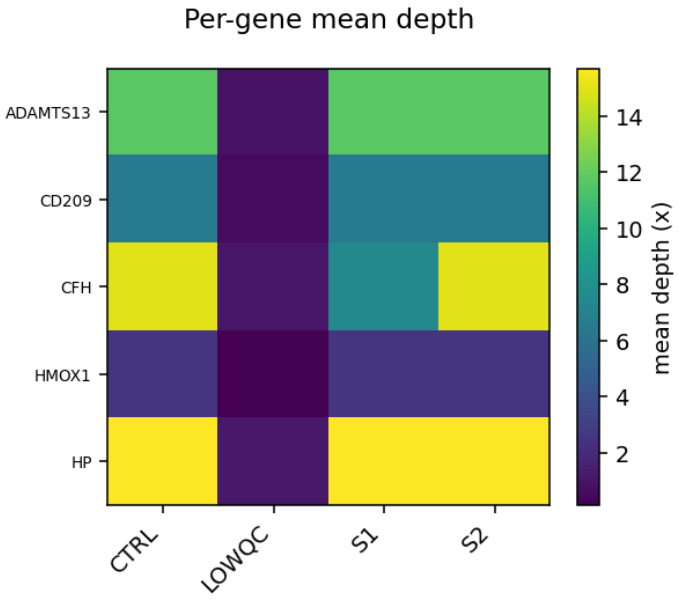


Figure 9.5: Mean coverage depth per target gene per sample; spot under-covered genes (e.g. CR1/CFH).

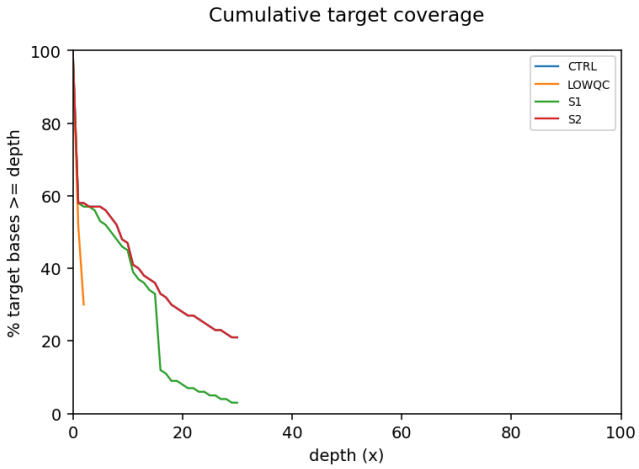


Figure 9.6: Fraction of target bases covered at >= each depth, per sample (sensitivity for calling).

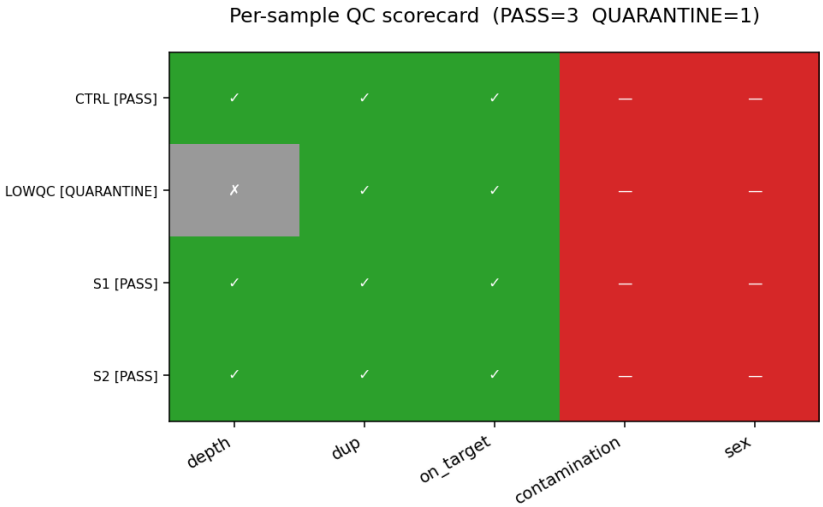


Figure 9.7: Per-sample QC checks (green=pass, red=fail, grey=not assessed); sample label shows the gate decision. Quarantined samples are kept and reported, not dropped.

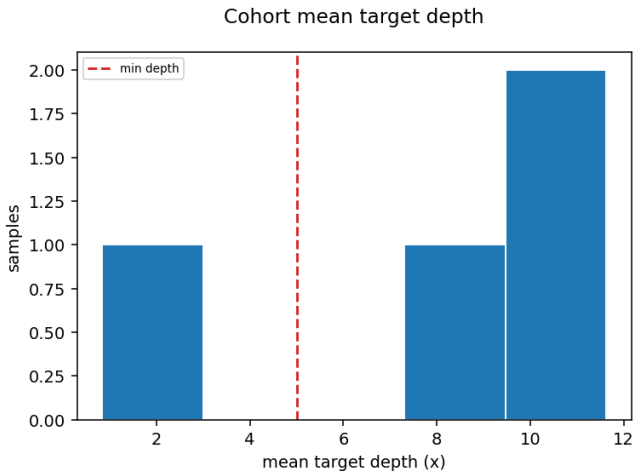


Figure 9.8: Distribution of per-sample mean target depth across the cohort vs the gate threshold.

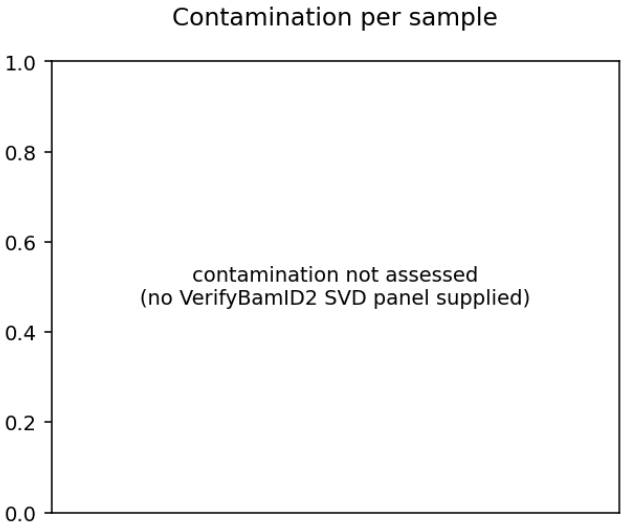


Figure 9.9: VerifyBamID2 FREEMIX contamination estimate per sample vs threshold (grey note if no SVD panel available).

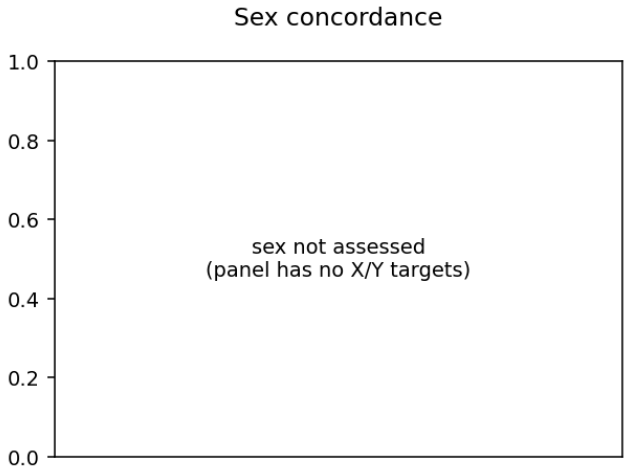


Figure 9.10: Declared vs inferred sex per sample (grey note if the panel has no X/Y targets to infer from).

9.3 9.3 Cohort QC dashboard & MultiQC (Stage 15)

`make_dashboard.py` builds `stage15_report/cohort_qc_dashboard.html` — a **self-contained** HTML embedding every captioned figure (base64) grouped by stage, plus the headline tables (resource manifest, QC scorecard, annotation landing, GIAB, burden). `multiqc_report.html` aggregates the per-tool outputs (FastQC, fastp, Picard, samtools, mosdepth).

10. Stage details & their plots

10.1 10.1 SNV/indel calling & filtering (Stages 6–7)

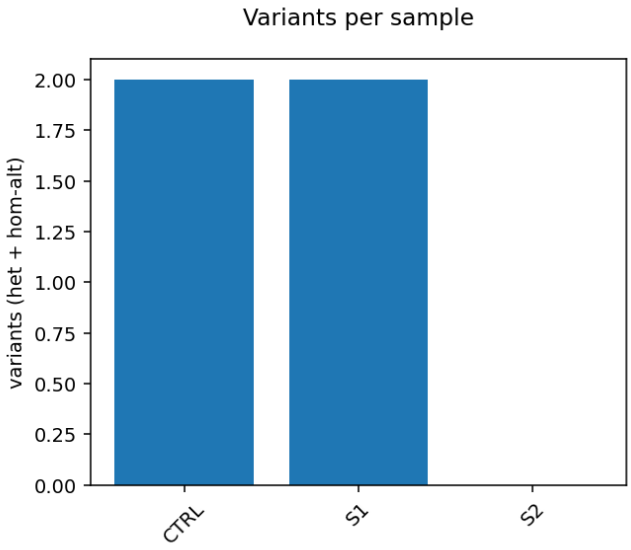


Figure 10.1: Per-sample count of non-reference genotypes in the joint callset (calling yield).

10.2 10.2 CNV, STR, paralog (Stages 8–10)

CNV calls (`cnv_calls.tsv`) carry a `cnv_callable` and `confidence` column from CaptureForge metadata — a call at a breakpoint-blind gene (CR1/CFH) is labelled `breakpoint_blind_low_confidence`, not silently trusted.

Paralog-aware flagging writes `INFO/PARALOG_GENE` and `INFO/PARALOG_CONF` into `paralog.annotated.vcf.gz` so each variant in a paralog region carries its confidence.

10.3 10.3 Annotation (Stage 11)

10.4 10.4 Cohort QC (Stage 12)

10.5 10.5 GIAB validation (Stage 13)

10.6 10.6 Burden (Stage 14)

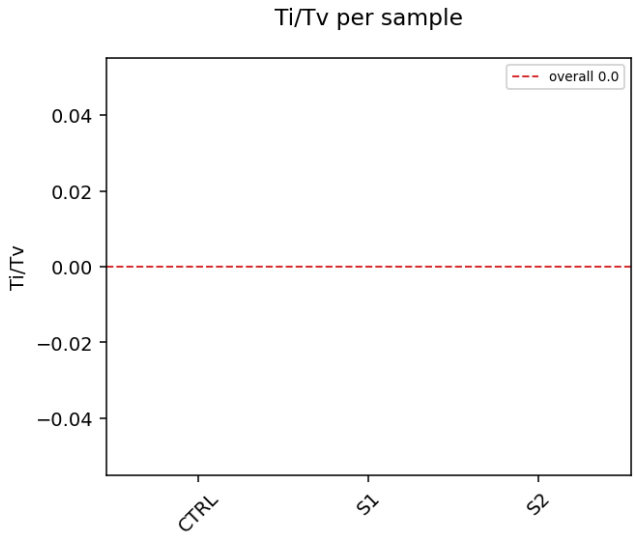


Figure 10.2: Transition/transversion ratio per sample; whole-exome ~3.0, smaller panels vary (calling quality).

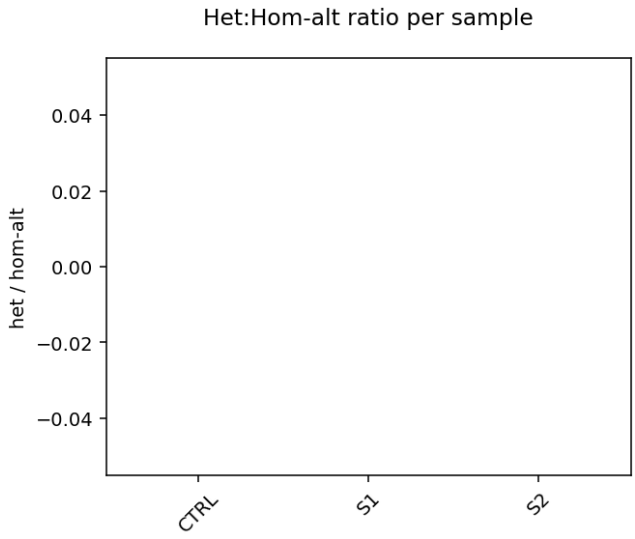


Figure 10.3: Heterozygous-to-homozygous-alt ratio per sample; extreme values flag sample-quality issues.

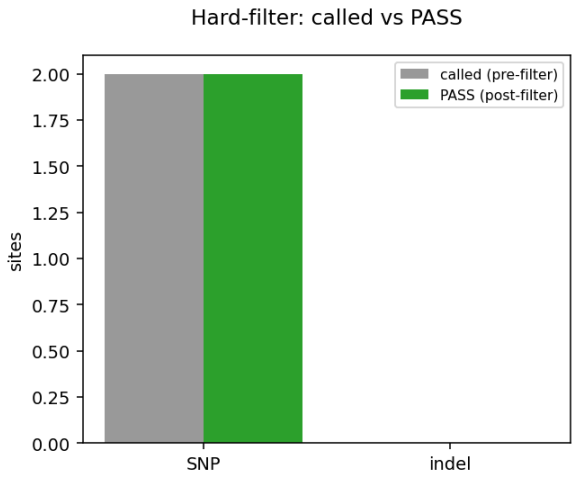


Figure 10.4: Variant counts before and after GATK hard-filtering, by type (sites retained).

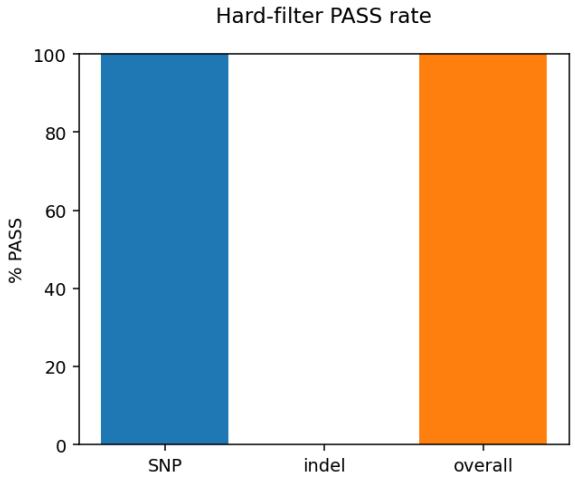


Figure 10.5: Fraction of called variants passing the GATK hard-filters, by type and overall.

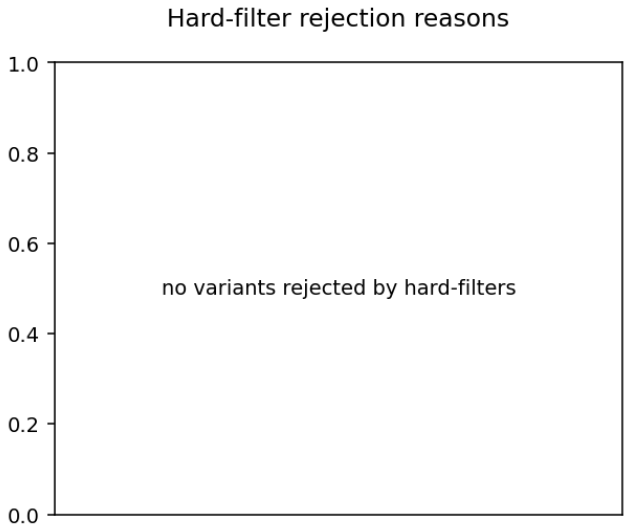


Figure 10.6: Number of sites rejected by each hard-filter criterion (which filter removed what).

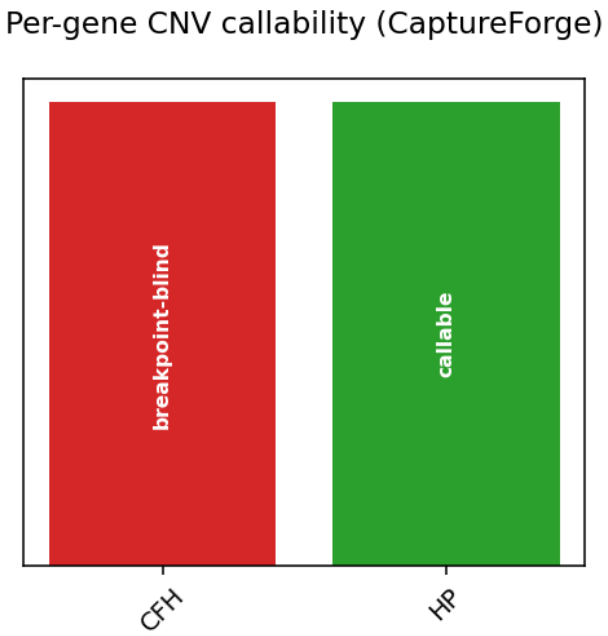


Figure 10.7: CNV-target genes coloured by CaptureForge callability: green=depth-callable, red=breakpoint-blind (low-confidence, e.g. CR1/CFH).

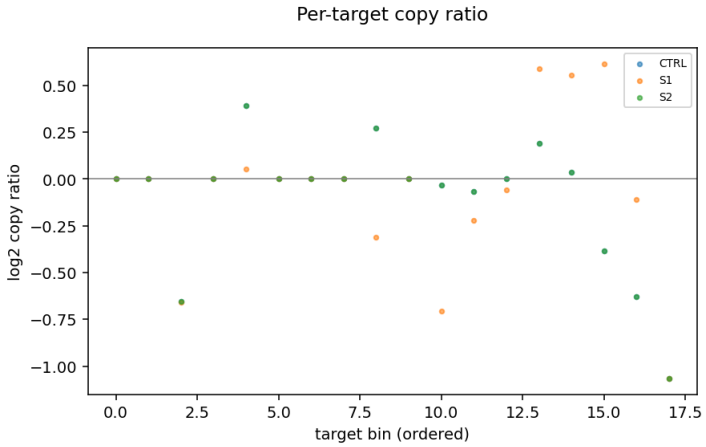


Figure 10.8: Per-target log2 copy ratio per sample; deviations from 0 indicate copy-number change.

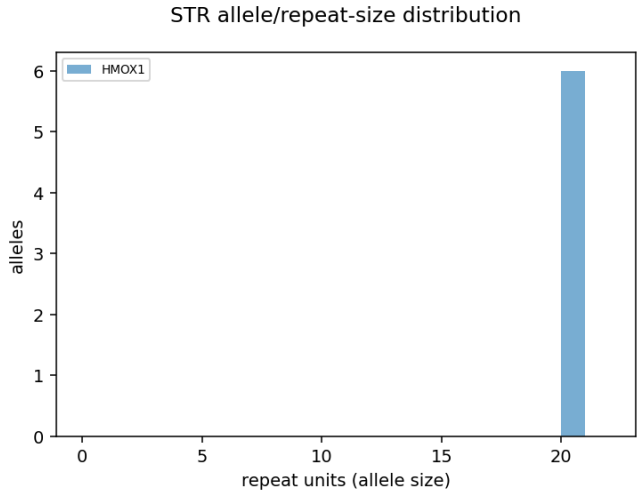


Figure 10.9: Distribution of genotyped repeat-unit counts (allele sizes) per STR locus.

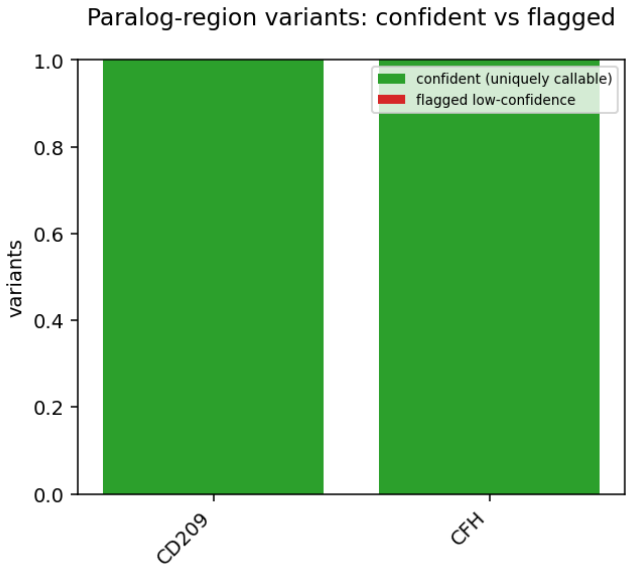


Figure 10.10: Variants in each paralog-ambiguous gene region, split into confident (uniquely callable, high MQ) vs flagged low-confidence (multimapping-ambiguous, low MQ).

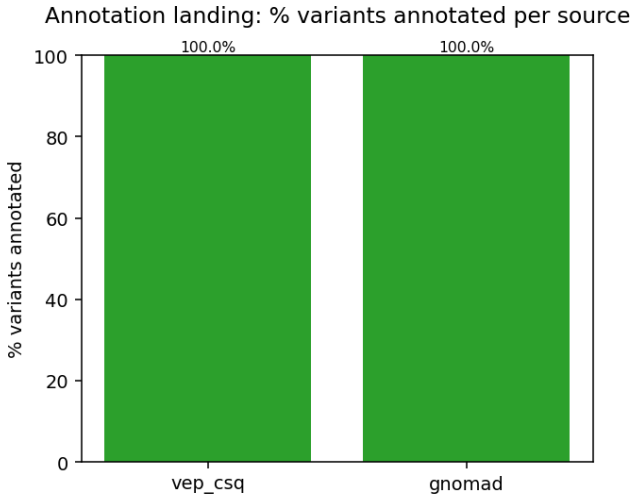


Figure 10.11: Fraction of callset variants that actually received each annotation (proof the annotation landed, not silently missed on a contig-name mismatch).

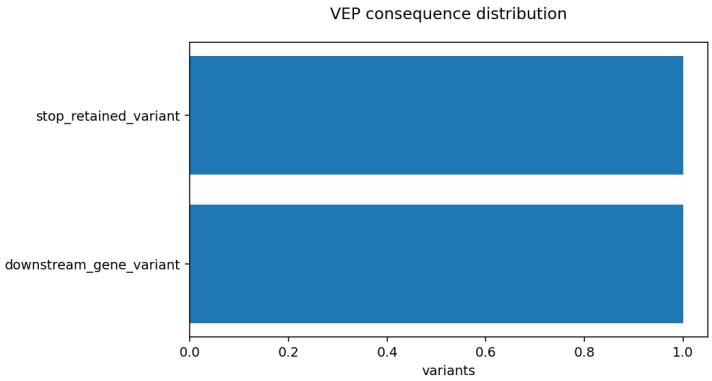


Figure 10.12: Most-severe VEP consequence per variant across the callset.

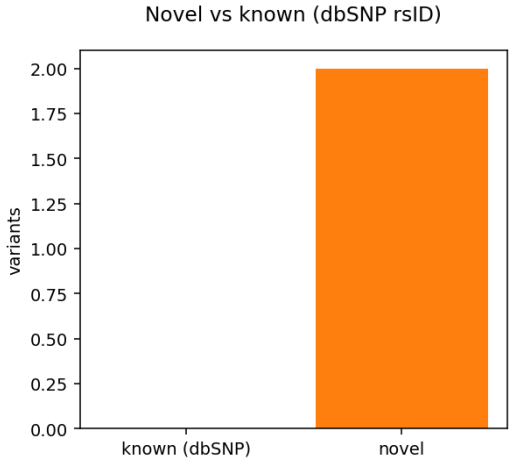


Figure 10.13: Variants with a dbSNP rsID (known) vs without (novel) — novelty rate.

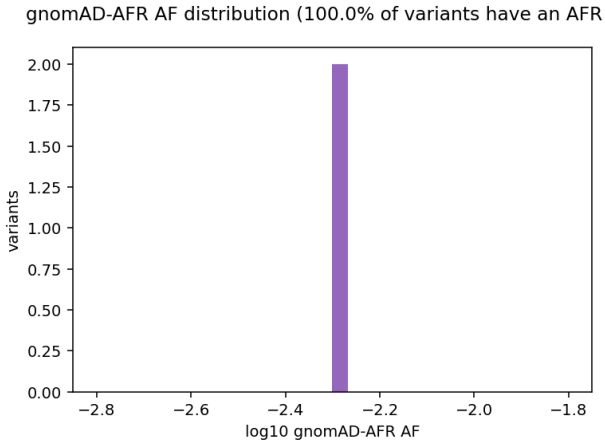


Figure 10.14: Distribution of gnomAD African-ancestry allele frequencies; title shows the % of variants carrying an AFR AF (key for rare-variant filtering / burden).

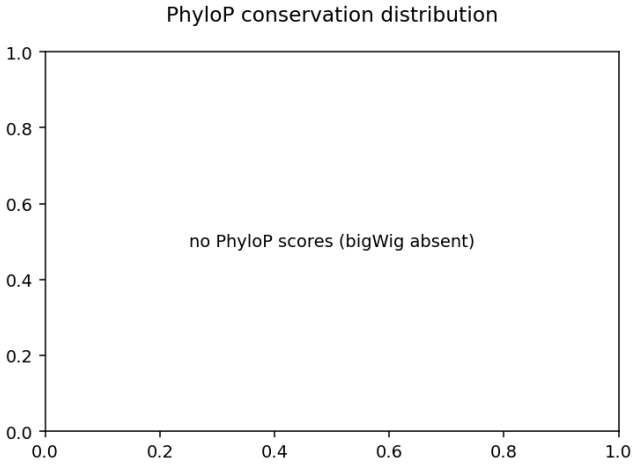


Figure 10.15: PhyloP conservation-score distribution; positive = conserved (likely functional).

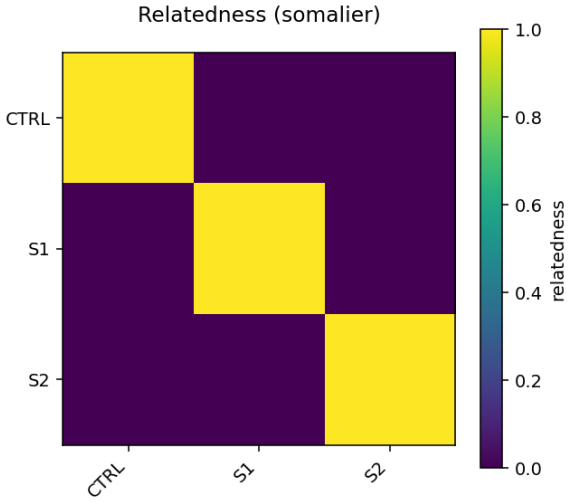


Figure 10.16: Pairwise somalier relatedness; synthetic test samples are near-identical so values run high (machinery proven) — a real cohort shows true relatedness structure.

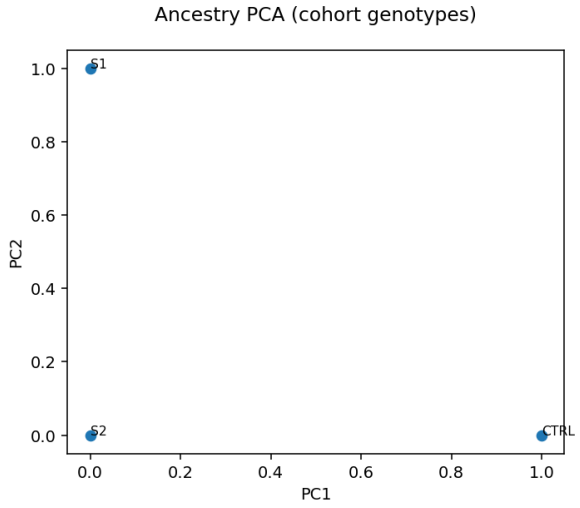


Figure 10.17: Genotype PCA of the cohort; on a tight panel few on-target sites give weak structure — real ancestry assignment uses somalier with the 1000G-labelled reference (off-target sites).

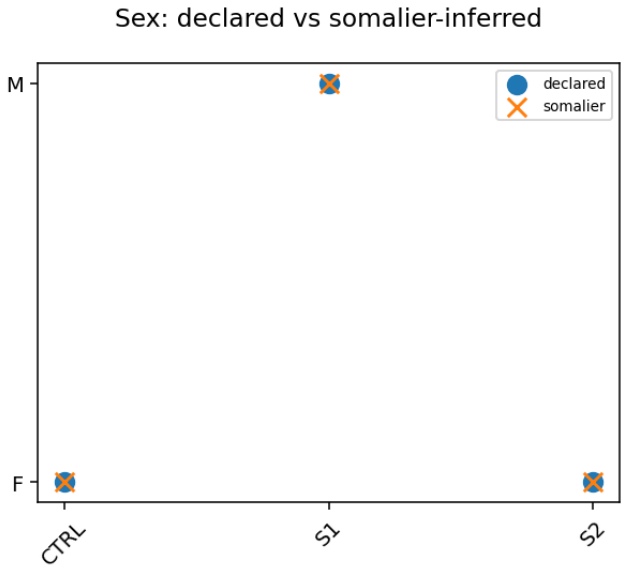


Figure 10.18: Declared vs somalier-inferred sex per sample (somalier infers from X/Y signal incl. off-target reads — the backstop for an autosomal panel with no X/Y targets).

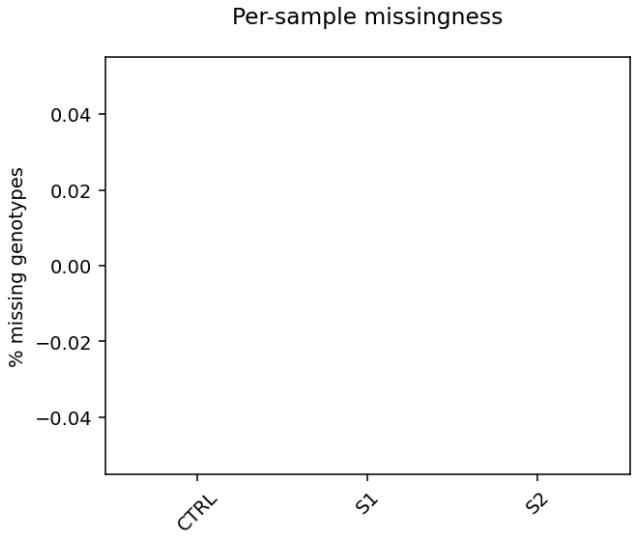


Figure 10.19: Per-sample fraction of no-call genotypes in the joint callset (sample quality).

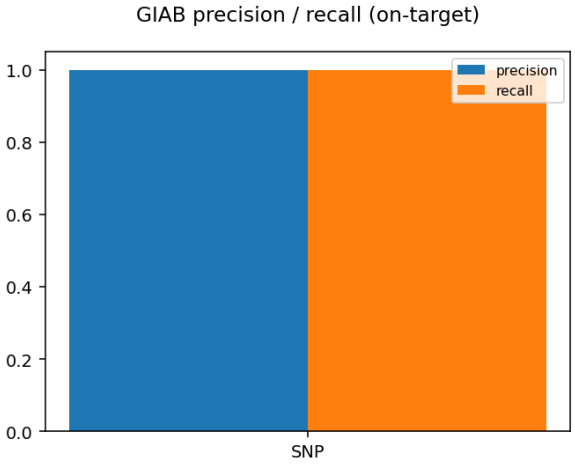


Figure 10.20: hap.py precision & recall vs GIAB truth, restricted to the panel BED (on-target). Synthetic test only proves the machinery; real numbers need a GIAB control + truth v4.2.1.

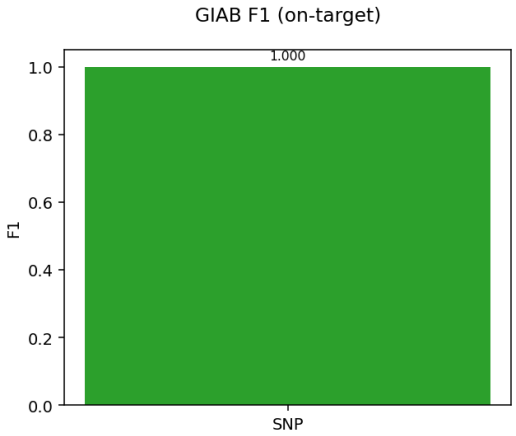


Figure 10.21: hap.py F1 per variant type vs GIAB truth, restricted to the panel BED (on-target).

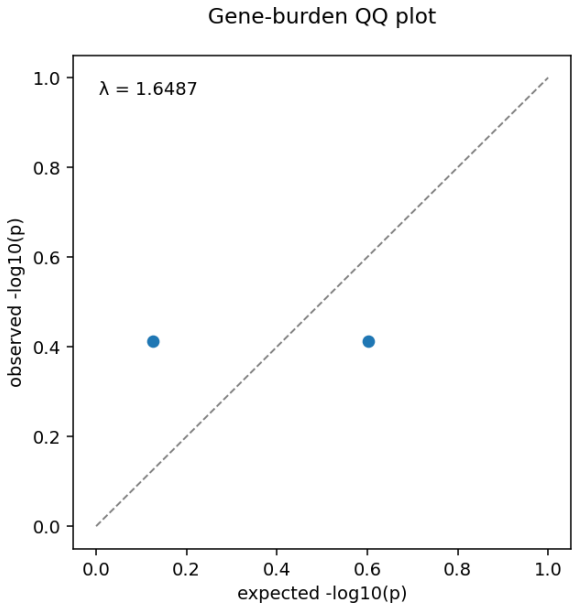


Figure 10.22: QQ plot of gene-burden p-values vs uniform expectation; lambda = genomic inflation. Small-N synthetic results prove the machinery only — not valid association.

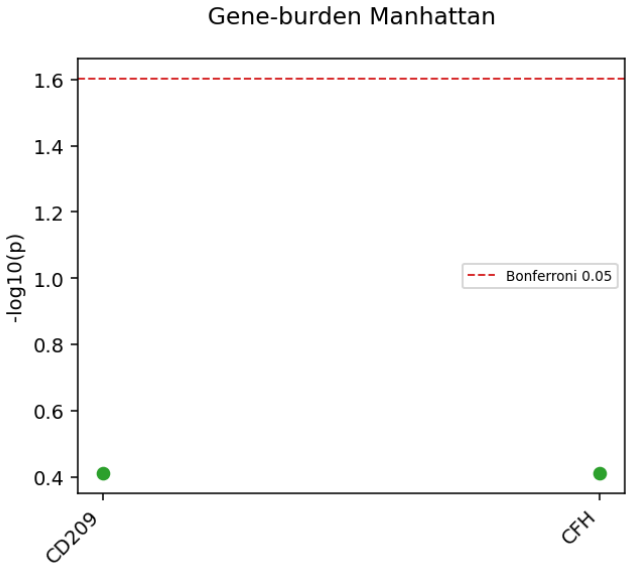


Figure 10.23: Per-gene burden significance (-log10 p) with a Bonferroni line; collapses rare+functional variants per gene. Validity depends on N, phenotype definition, ancestry matching.

11. Outputs explained

Output	Location	File + notes
Annotated joint VCF	stage11_annotation/	annotated.vcf.gz — VEP CSQ + gnomAD_* + ClinVar_* + PhyloP + paralog flags
Per-variant TSV	stage11_annotation/	variants.flat.tsv — flattened (columns below)
Filtered joint VCF	stage7_filter/	joint.filtered.vcf.gz — FILTER = PASS or hard-filter name
CNV calls	stage8_cnv/	cnv_calls.tsv — with cnv_callable + confidence columns
STR genotypes	stage9_str/	str_calls.tsv — repeat units, read support, call rate
Paralog VCF	stage10_paralog/	paralog.annotated.vcf.gz — INFO/PARALOG_GENE, PARALOG_CONF
Burden results	stage14_burden/	burden_results.tsv — engine column stamps the method
Cohort QC	stage12_cohortqc/	cohort_qc.json — relatedness, sex, missingness, PCA
GIAB metrics	stage13_giab/	giab_metrics.json — precision/recall/F1, on-target
Provenance	stage15_report/	provenance.json — versions, resources, quarantine, PC gate
Dashboard	stage15_report/	cohort_qc_dashboard.html — all figures + tables

Per-variant TSV columns (`variants.flat.tsv`): `chrom`, `pos`, `ref`, `alt`, `rsID`, `filter`, `gene`, `consequence`, `gnomAD_AF`, `gnomAD_AF_afr`, `ClinVar_CLNSIG`, `PhyloP`, `paralog_gene`, `paralog_conf`.

Burden results columns (`burden_results.tsv`): `unit`, `kind`, `engine`, `n_carriers`, `case_carriers`, `control_carriers`, `chi2`, `p` (the `engine` column means a `collapse` result can never be mistaken for a regenie association when the file is read on its own).

12. Interpreting results

Machinery-proven vs needs-real-cohort. On the synthetic `test` cohort the pipeline *runs* and the *invariants fire*, but the *numbers* are not data quality. Treat as machinery-proven (not real): on-target/enrichment/uniformity, Ti/Tv & het:hom, GIAB precision/recall, all burden association (small N), and relatedness/ancestry structure.

CaptureForge confidence labels. Read CNV calls *through* their callability label: depth-callable genes (e.g. GYPC, HP) are trustworthy; breakpoint-blind genes (CR1, CFH) are `breakpoint_blind_low_confidence` — treat their CNV calls cautiously. Paralog-region variants are split **confident** (uniquely callable, high MQ) vs **flagged** (multimapping ambiguity) — confirm flagged calls orthogonally.

Burden validity. Small cohorts show **genomic inflation** (high λ in the QQ plot) — not signal. The **ancestry-PC gate** drops unstable PCs and runs **unadjusted** (stated in `provenance.json` and the burden summary); an unadjusted run is a deliberate, declared choice, not a silent one. Burden validity depends on N, phenotype definition, and ancestry matching — CallForge does not imply publication-ready association on a small cohort.

GIAB. Precision = fraction of called variants that are true; recall = fraction of truth recovered; both **on-target** (panel-BED-restricted). Genome-wide scoring would distort a panel's measured sensitivity.

13. Real-run checklist

Before the production cohort:

- ☐ Demultiplexed FASTQs + sample sheet (with `phenotype` for burden).
- ☐ **Genome-wide gnomAD (AFR), ClinVar, dbSNP, PhyloP** — each must pass the scope gate. dbSNP is needed in **two** places (annotation rsIDs + BQSR known-sites).
- ☐ BQSR known-sites via `--known_sites` (dbSNP + Mills + 1000G indels).
- ☐ **1000G GRCh38 somalier sites** via `--somalier_sites`.
- ☐ GIAB control in the sheet + truth v4.2.1 (`--giab_control_id`, `--giab_truth_vcf/bed`).
- ☐ **Confirm ancestry PCs are stable** before using them as burden covariates (the gate enforces this; if unstable it runs unadjusted and says so).
- ☐ **Validate-before-production engines**: DeepVariant+GLnexus, regenie, SKAT-O/STAAR.
- ☐ Re-generate the genome-wide gnomAD panel slice on the HPC (fast network).
- ☐ Revisit GATK hard-filter thresholds for targeted data.

14. Troubleshooting

Symptom	Cause / fix
`container engine … is enabled but --container_image is unset`	wire the `.sif`/tag: `--container_image /path/callforge.sif` (or omit the engine profile). Prevents the silent host-fallback bug
`SLURM executor selected but --slurm_partition is unset`	set `--slurm_partition <queue>` (no default); find one with `sinfo -s`
Task fails "command not found" / "No such file" on a ref under a container	bind the ref's root: `export SINGULARITY_BIND=/hpc` (APPTAINER_BIND on Apptainer) or `--bind_paths /hpc` — auto-mounts cover only `$HOME`/`tmp`/CWD
`reference_invariant FAIL` (alt contigs / chr-style mismatch / interval overrun)	use the no-alt primary assembly; make the target BED's chr-prefix style match the reference
Scope-gate FAIL on a DB	region-subset DB — supply a genome-wide replacement (message names it)
Annotation values missing	chr-naming mismatch — handled by reconciliation; `verify_annotation.py` fails loud if not, check the manifest naming
VEP "CacheDir" error in gtf mode	do not pass `--offline` with `--gtf` (cache mode only)
mosdepth "exec format error" (arm64)	use mosdepth $\geq$0.3.11 or `CONDA_SUBDIR=osx-64`
`grep -P` errors in fetch script (macOS)	BSD grep lacks `-P`; the shipped script uses `awk` (keep it in sync)
No samples reach calling	all quarantined — read `qc_quarantine.txt`; relax thresholds only with justification
Unstable ancestry PCs	the gate drops them and runs unadjusted (recorded); do not force them
Truncated `.bgz` DB	BGZF-EOF check flags it `truncated` — re-fetch (resumable)
Hard-filter over-removes on a panel	tune `--snp_filter_expr/--indel_filter_expr` (defaults are WGS/WES)

15. Extending CallForge

- **New annotation databases**: add them to a vcfanno TOML (`--vcfanno_toml`); the scope gate + landing check apply.
- **New organism**: `--species` + VEP cache + vcfanno TOML; human DBs skip (§6.6).
- **Engine choices**: §8.8 — note the validate-before-production engines.
- **Roadmap (not yet validated)**: end-to-end runs of DeepVariant+GLnexus, regenie, and SKAT-O on a real cohort; VQSR for large cohorts; manifest-driven BQSR known-sites.

16. Reproducibility & provenance

Every run writes `stage15_report/provenance.json`: pipeline version + git commit, the reference + invariant statement, the resource manifest (status/build/md5/AFR), QC thresholds + pass/quarantine lists with reasons, the burden status + **ancestry-PC gate decision**, and pointers to the pinned `env/*.yml` and the pinned VEP/hap.py containers. Pin Nextflow via `manifest.nextflowVersion` and tool versions via the env files; cite the exact reference and database releases recorded in the manifest.

17. Glossary & FAQ

gVCF — per-sample genomic VCF with reference-confidence blocks, the input to joint genotyping. **PoN** — panel-of-normals (CNVkit reference from QC-pass samples). **FREEMIX** — VerifyBamID2 contamination estimate. **FOLD_80** — uniformity penalty (1.0 = uniform). λ **(lambda)** — genomic inflation factor. **Burden test** — gene/gene-set aggregation of rare variants vs phenotype.

FAQ.

- *Do I need CaptureForge to run CallForge?* No — you can supply `--target_bed` + `--gene_metadata` directly; but the per-gene labels (CNV callability, paralog, burden groups) come from CaptureForge metadata and enrich the analysis.
- *What if I have no phenotype?* The pipeline runs through the annotated callset and skips burden with a message.
- *Why did a sample disappear from the callset?* It was quarantined — see `qc_quarantine.txt`.

18. Citation & the CaptureForge ↔ CallForge pairing

Cite via `CITATION.cff` in the repository root. CallForge is the **analysis counterpart to CaptureForge**: CaptureForge designs the panel and records per-gene design knowledge; CallForge consumes that handoff and produces the analysed, annotated, QC'd callset and burden layer. Run them as a pair for a panel: design → sequence → **CallForge**.